

PreAct: Computer-Using Agents that Get Faster on Repeated Tasks

Bojie Li
Pine AI

Abstract

Computer-using agents drive real software by reading the screen and issuing clicks and keystrokes, but they solve every task from scratch. An agent that adds a contact to the address book today reads the screen, reasons about each tap, and acts; asked for the same task tomorrow, it does all of that again and pays the same cost a second time. We present PreAct, which lets such an agent get faster on the tasks it has done before by acting from a pre-compiled program instead of re-deriving each step. The first time the agent succeeds, PreAct turns the run into a small state-machine program—states that check the screen, transitions that act—and saves it. When the same task comes up again, PreAct runs the program directly instead of the agent, roughly an order of magnitude more cheaply, and only calls the agent back when a check fails.

Reusing programs this way has a catch: a stored program can run all the way to its last step and still not accomplish the task, leaving the contact unsaved. PreAct guards against this by re-running each freshly compiled program from a clean state and keeping it only when an independent evaluator confirms the task was solved. Across three different environments—a mobile, a desktop, and a web benchmark—this check is what separates repeated runs that improve from ones that slowly degrade as faulty programs pile up; on the web benchmark, where most programs cannot be re-verified, it limits the damage rather than reversing it. A simple fallback, exploring a task afresh whenever no stored program fits, brings PreAct level with a strong record-and-replay baseline. We also report what did not matter: the wording of the agent’s prompts and a set of runtime guardrails make no measurable difference, and a plain embedding retriever chooses which program to reuse about as well as a language model does.

Code: <https://github.com/bojieli/PreAct>

Website: <https://01.me/research/PreAct/>

1 Introduction

Computer-using agents—models that operate ordinary software by reading the screen and issuing clicks and keystrokes [57, 1]—are now good enough to handle everyday chores like booking a meeting or filing an expense. They are also wasteful. Consider a task from the AndroidWorld benchmark [36]: *add a new contact named “Emilia Gonzalez” with a given phone number*. The agent reads a screenshot, decides where to tap, acts, and repeats, spending half a dozen rounds of vision-and-language inference to walk through the contacts app. Ask for the

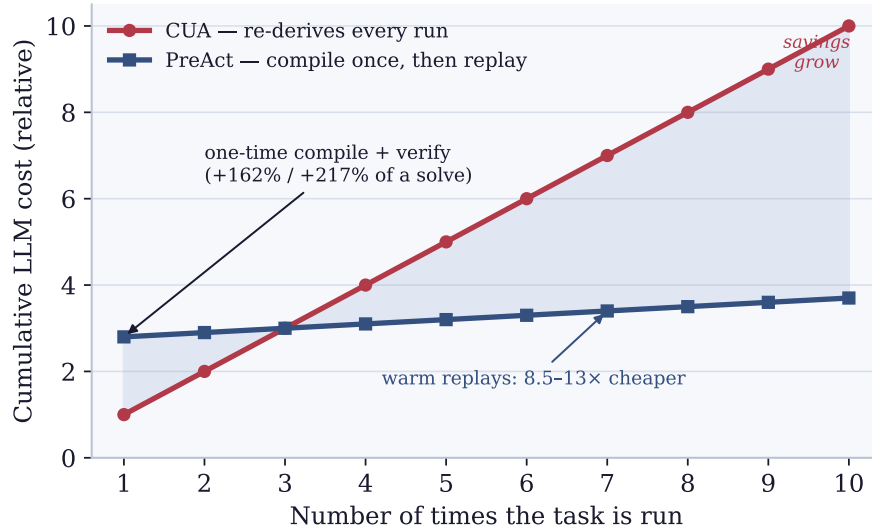


Figure 1. The cost of doing the same task twice. A standard computer-using agent re-derives a familiar task on every run, so its cumulative cost grows with the number of runs (red). PreAct pays a one-time premium to compile and check the first success (the bump at run 1), then replays the stored program on every later run at 8.5–13× lower cost (blue). The more often a task recurs, the more this saves. Cost is normalised to one full agent solve; the speedup and the one-time overhead are measured, not illustrative.

same task tomorrow and it starts over. The screen has not changed and neither have the taps, but the agent kept nothing it could simply run again, so it pays the full cost a second time.

This is not how people work. Someone opening Photoshop for the first time, or a traveler in China reaching for Alipay or WeChat Pay to settle a bill, is painfully slow at first—reading every menu, hunting for the right button. After a few repetitions they are nearly as fluent as a local: the deliberate, perception-heavy reasoning of the first attempt has been internalized into an almost automatic routine. The animating question behind PreAct is whether a computer-using agent can acquire the same continual-learning capability—not merely solving a task it has seen before, but solving it *faster and more cheaply* the second time, much as a practiced human does. This efficiency dimension is largely absent from how agents are evaluated: most benchmarks score only whether a task *succeeds*, not how much perception and reasoning a repeated success costs, so an agent that re-derives every task from scratch looks no worse than one that has learned to skip the deliberation. We treat repeated-task efficiency as a first-class outcome and measure it directly (Figure 1).

Several lines of work try to reuse past behaviour, but each keeps a model in the execution loop or stores something it cannot reliably re-run. Skill libraries such as SAGE [25] save parameterised behaviours, yet a language model still carries them out at run time, so most of the per-step cost remains. Compilation systems [43, 63] turn behaviour into deterministic code, but they target narrow business logic, or build a program by crawling an application rather than from an actual successful run. Record-and-replay systems [34, 11, 4] cache the sequence of actions from one run and replay it, falling back to the agent when the cache misses; the cached trace has no way to tell whether each step really landed, no room for branching, and no means of improving itself.

PreAct takes a different position: the thing the agent stores is exactly the thing it later runs. The first time the agent finishes “add Emilia Gonzalez,” PreAct turns the run into a small state machine. Each state carries a check on the screen—the contact form is open; the name field now reads the value we typed—and each transition carries an action, such as tapping Create contact or typing the first name. The next time a matching task arrives, PreAct retrieves that state machine and walks it: at every state it confirms the screen looks right before doing anything, and at every transition it performs the action. No language model runs during a successful replay, which is where the order-of-magnitude saving comes from. When a check does fail—the app has changed, an unexpected dialog appears—PreAct hands control back to the full agent, which finishes the task and produces a fresh program. Over time the store grows on its own: the code that drives this loop is fixed, and what accumulates is a library of programs the agent can run directly (Figure 4).

Storing programs blindly is dangerous, and this is the part that took us the longest to get right. A compiled program can replay all the way to its final step and still leave the task undone: it taps through the contact form and presses Save, but a stale field meant the name was never entered, so no contact exists. An agent that trusts such a program will fail the same way every time it reuses it, and the more it runs the worse it becomes. PreAct’s answer is to check a program before keeping it. Right after compiling, it resets the environment, runs the new program from scratch, and asks the benchmark’s own evaluator whether the task was actually solved; only then is the program allowed into the store. This one check is the difference between a system that gets better with use and one that quietly decays.

We test PreAct on three platforms—a phone (AndroidWorld), a desktop (OSWorld), and a website (WebArena)—where warm replays run 8.5–13 $\times$ cheaper than re-deriving the task from scratch. Turning the check on or off, with a fresh corpus or one the agent has already built, gives a consistent picture: with the check, repeated runs get better; without it, they get worse as faulty programs collect. The gate is worth 1.75–2.6 tasks across the cold $\rightarrow$ warm transition, pointing the same way on every platform (pooled $p \approx 10^{-4}$). A second, smaller idea matters too—when no stored program fits a task, exploring it afresh rather than giving up keeps PreAct level with a strong record-and-replay baseline (Muscle-Mem; the warm-run difference is statistically indistinguishable, $p \approx 0.84$). We are also careful to report what did *not* change the numbers, since it is easy to credit the wrong thing: the exact wording of the prompts, a handful of hand-written runtime guardrails, and whether a language model or a plain embedding retriever (which matches the agentic selector at 100% vs. 75.6% retrieval) picks which stored program to reuse all made little or no difference. What carries the result is the design itself—store the program you run, and never keep one you have not checked.

2 Related Work

PreAct sits at the intersection of six literatures: computer-using agents, the reason–act loop they run, skill and memory systems that try to reuse past behaviour, code as an agent representation, world models that learn environment dynamics, and systems that compile or replay workflows. We survey each in turn, then (Table 1, Figure 2) locate PreAct relative to the closest prior work.

Computer-using agents. A rapidly growing line of work builds agents that operate real software through its graphical interface—clicking, typing, and scrolling—rather than through

bespoke APIs [1]. Early web agents learned from demonstrations or reinforcement on simulated sites [30, 55, 13], and benchmarks such as Mind2Web [10], WebArena [64], VisualWebArena [20], and WebVoyager [17] pushed toward realistic, open-ended websites. A parallel thread grounds vision-language models directly on screen pixels and accessibility trees: SeeAct [62], CogAgent [18], SeeClick [8], and UI-TARS [35] learn to localize and act on UI elements. The same paradigm now spans mobile [59, 46, 37, 36], desktop and OS control [31, 51, 58, 3, 52], and software engineering [54, 19], with reinforcement learning [2] and general-assistant benchmarks [29] measuring progress. PreAct is agnostic to which such agent fills its CUA-fallback slot; we use AndroidWorld’s T3A and Anthropic’s Computer-Use API, but the harness wraps any of them.

The reason–act loop and its cost. Almost all of these agents follow the observe–reason–act paradigm popularized by ReAct [57], layered on chain-of-thought and search-style reasoning [50, 56, 48], tool use [38], self-reflection [39], and autonomous task loops [40, 41]. This loop is exactly what makes repeated execution expensive: every step pays for fresh perception and reasoning even when the task is one the agent has solved before [26]. PreAct attacks this cost directly by replacing the loop, on familiar tasks, with deterministic graph execution.

Skill libraries and experiential memory. A second response to repetition is to remember. Skill systems accumulate reusable behaviours—Voyager grows a library of code skills [45], SAGE and Skill-Pro parameterize learned skills [25, 28], and ExpeL distills experience into rules [61]—but each skill still runs inside an LLM-bound loop at execution time. More broadly, agent-memory systems store and retrieve past interactions: generative-agent memory streams [33], operating-system-style memory [32], and production memory stacks [9, 53] (surveyed in [60]), typically consulted by retrieval-augmented generation [21]. PreAct’s corpus is a memory in this sense, but what it stores is *executable*: recall and action are the same step. Concurrent work pushes the same executable-memory principle—store memory as code an interpreter can run, rather than text consulted by retrieval—into *personalized* agents: User-as-Code [22] represents a user’s accumulated state as typed Python objects and the rules over it as executable constraints, so representing the user and reasoning about the user share one medium. The two papers differ in what the code *is*: there it is typed state plus constraint predicates queried over a user model; here it is a verified state-machine *action program* whose execution is itself the recall. Scaling such reuse, rather than re-deriving behaviour, is consistent with the “bitter lesson” that general mechanisms outrun hand-engineered ones [42].

Code as the agent’s action and representation. A third thread expresses agent behaviour as code. PAL, Program-of-Thoughts, and CoCoGen offload reasoning to executable programs [12, 7, 27]; Code-as-Policies and CodeAct make code the agent’s action space [24, 47]; Chain-of-Code emulates execution [23]; and TroVE and LATM induce reusable, verifiable toolboxes from solved tasks [49, 5]. PreAct shares the conviction that code is the right substrate for reliable reuse, but its compiled artifact is a *verified state machine executed directly*, not a flat script regenerated and re-interpreted by an LLM each time.

Internalized vs. externalized world models. A fourth thread learns the environment’s *dynamics*—a model that predicts the next observation, or a tool’s response, from the current state and a candidate action. Classical world models learn these dynamics in network parameters and plan inside the learned simulator [14, 15]; the idea has since migrated to language agents that use the LLM itself as a world model to predict action outcomes for planning [16], including

Table 1. PreAct vs. related approaches.

System	Memory representation	Replay fidelity	Fallback path	Self-extension
Skill libraries (e.g. Voyager)	Code skills	LLM-bound (skill called by agent)	LLM	Grows/self-verifies skill library
Muscle-Mem	Linear tool calls	Direct	Agent on cache miss	Append cache only
AgentRR	Multi-level experiences	Indirect	LLM	Append only
Workflow-Use	Linear scripts	Direct	Agent	Append only
ActionEngine	State machine via crawl	Flat-Python generated	None	Re-crawl from scratch
PreAct	State machine	Runs the state machine directly	CUA + recompile under verify-gate	Verified corpus growth + dedup-replacement

for web navigation, where an agent learns environment dynamics to anticipate the effect of a click before committing to it [6]. PreAct can be read as a world model of a different kind. Each stored program is an *externalized, verifiable* model of one task’s dynamics—its state predicates are checkable claims about what the screen will be, and its transitions are the actions that move between those states—rather than dynamics baked into weights. The difference is where the learning lives and whether it can be trusted: an internalized world model predicts transitions implicitly and can hallucinate them with no way to check against the live environment, whereas PreAct’s verify-before-store gate (§3.3) is exactly a test that the learned dynamics still hold before a program is trusted, and a program can be inspected, replaced, or repaired without retraining. The price is generality: these are task-specific models grounded in concrete selectors, not one dense dynamics model, and they do not transfer across task families (§5.5).

Compiling and replaying workflows. Closest to PreAct are systems that compile or cache concrete workflows. Robotic process automation has long replayed recorded UI scripts deterministically [44]; Compiled-AI and ActionEngine compile agent behaviour into deterministic code, the latter generating flat Python from a state machine built by untargeted crawling [43, 63]; and concurrent record-and-replay systems—Muscle-Mem [34], AgentRR [11], and Workflow-Use [4]—cache a linear action sequence and fall back to the agent on a cache miss. We compare against these directly. They share two gaps that PreAct closes: the stored artifact has no per-state verification against the live UI, and it does not improve itself—it is appended, never refined or revalidated. Table 1 summarizes the comparison.

Running the state machine directly. What distinguishes PreAct is the second column (it runs the state machine directly) combined with the fourth (recompile under the verify-gate). ActionEngine builds a state machine but emits flat Python from it; once the script is generated the state machine is thrown away, and what actually runs is that script. PreAct instead executes the state machine itself: each state’s verification predicate is evaluated against the live UI, and each transition is dispatched to the underlying action backend (XPath click, JSON action, etc.). This direct execution enables (a) per-state verification and graceful fallback, (b) in-place patching when a transition fails, and (c) the verify-before-store gate (which we will argue is empirically necessary for monotonicity).

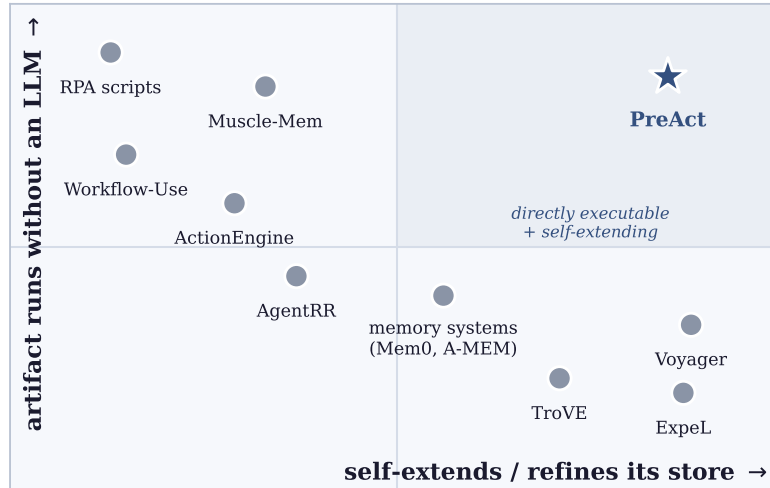


Figure 2. Where PreAct sits, along the two properties it is designed to combine. Many prior systems achieve one or the other. *Record-and-replay* systems (Muscle-Mem, Workflow-Use) and classical RPA run their stored artifact *without* an LLM, but only append it (top-left). *Skill and memory* systems—Voyager, TroVE, ExpeL, Mem0/A-MEM—genuinely self-extend, and some (Voyager, TroVE) even self-verify what they add, but they invoke it *through* an LLM at run time (bottom-right). PreAct’s distinction is the *combination*: a directly-executable artifact (no LLM in the loop at replay) grown through verified, in-place refinement. Axes are qualitative.

What grows with experience. The deeper distinction concerns *what grows* as the agent gains experience. In Muscle-Mem, AgentRR, and Workflow-Use, the cache or experience store is append-only: every captured trajectory adds a new entry without affecting the existing ones. In ActionEngine, the state machine is built once via crawling and the flat Python is regenerated wholesale on each rebuild; there is no notion of incremental refinement. PreAct’s compile-extend-replace loop, by contrast, allows the corpus to *mutate*: a fresh compile with the same dedup signature replaces an older program if it passes the verify-gate. The corpus does not just accumulate—it refines its representation of repeated tasks as the agent encounters them across different parameter values, environments, and sessions. This is the property we mean by “self-extending executable code corpus” (§1): not an append-only cache, but a mutable, verified, directly-executable code library.

3 System Architecture

3.1 What the agent stores: a state-machine program

We start from the artifact, because in PreAct the agent runs that artifact directly rather than regenerating a script from it. Listing 1 is an *actual* program from the corpus, compiled while the agent ran the `ContactsAddContact` task from the `AndroidWorld` benchmark [36]—the trace was the instance “Create a new contact for Emilia Gonzalez.” It is a small graph, drawn in Figure 3: each *state* carries a verification predicate (an accessibility-tree pattern that must hold on the live screen for the agent to believe it is in that state) and each *transition* carries an action. The concrete values typed during the trace are lifted to *parameters*—here `first_name`, `last_name`, `phone_number`, listed in the metadata—so this single program serves every future

“add a contact” request by binding the parameters to each new contact’s values.

```
{
  "metadata": {
    "program_id": "ab4390a9-1902-47a1-821b-ebddc7f4010a",
    "task_description": "Create a new contact for Emilia Gonzalez. Their number is +14240925675.",
    "application_context": "com.google.android.contacts",
    "parameters": ["first_name", "last_name", "phone_number"]
  },
  "states": [
    { "id": "contacts_app_open",
      "verification": {"type": "expect_element", "xpath": "resource_id=com.google.android.contacts:id/floating_action_button", "timeout_ms": 5000},
      "description": "Contacts app is open showing the main list with the Create-Contact FAB visible" },
    { "id": "create_contact_form",
      "verification": {"type": "expect_element", "xpath": "class=android.widget.EditText&&hint=First name", "timeout_ms": 3000},
      "description": "New-contact creation form is open with the First name field visible" },
    { "id": "first_name_entered",
      "verification": {"type": "expect_element", "xpath": "class=android.widget.EditText&&hint=Last name", "timeout_ms": 2000},
      "description": "First name has been entered; Last name field is available" },
    { "id": "last_name_entered",
      "verification": {"type": "expect_element", "xpath": "class=android.widget.EditText&&hint=Phone", "timeout_ms": 2000},
      "description": "Last name has been entered; Phone field is available" },
    { "id": "phone_entered",
      "verification": {"type": "expect_element", "xpath": "resource_id=android:id/text1&&text=Mobile", "timeout_ms": 2000},
      "description": "Phone number has been entered; phone-type selector (Mobile) is visible" },
    { "id": "phone_type_selected",
      "verification": {"type": "expect_element", "xpath": "resource_id=com.google.android.contacts:id/toolbar_button&&text=Save", "timeout_ms": 2000},
      "description": "Phone type Mobile selected; Save button is visible in the toolbar" },
    { "id": "contact_saved",
      "verification": {"type": "terminal_state", "timeout_ms": 5000},
      "description": "Contact has been saved successfully and the task is complete" }
  ],
  "transitions": [
    { "from_state": "contacts_app_open", "to_state": "create_contact_form",
      "action": {"type": "action_navigate", "text": "Contacts"} },
    { "from_state": "contacts_app_open", "to_state": "create_contact_form",
      "action": {"type": "action_click", "target": "resource_id=com.google.android.contacts:id/floating_action_button"} },
    { "from_state": "create_contact_form", "to_state": "first_name_entered",
      "action": {"type": "action_type", "target": "class=android.widget.EditText&&hint=First name", "parameter_name": "first_name"} },
    { "from_state": "first_name_entered", "to_state": "last_name_entered",
      "action": {"type": "action_type", "target": "class=android.widget.EditText&&hint=Last name", "parameter_name": "last_name"} },
    { "from_state": "last_name_entered", "to_state": "phone_entered",
      "action": {"type": "action_type", "target": "class=android.widget.EditText&&hint=Phone", "parameter_name": "phone_number"} },
  ]
}
```

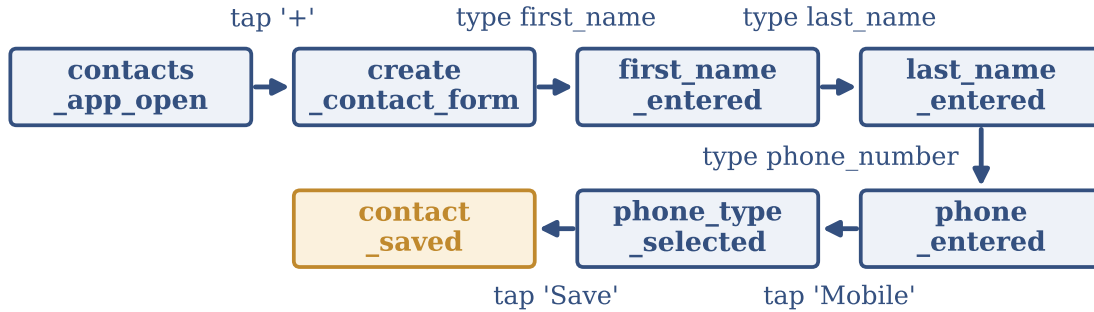


Figure 3. The compiled “add a contact” program of Listing 1 drawn as a state machine (read top row left-to-right, then bottom row right-to-left). Each box is a state—its verification predicate, which must match the live screen, is given in Listing 1—and each arrow is a transition holding an action. The harness executes this graph directly; it is not regenerated into a flat script.

```

{ "from_state": "phone_entered", "to_state": "phone_type_selected",
  "action": { "type": "action_click", "target": "resource_id=android:id/text1&&text=Mobile"
} },
{ "from_state": "phone_type_selected", "to_state": "contact_saved",
  "action": { "type": "action_click", "target": "resource_id=com.google.android.contacts:
id/toolbar_button&&text=Save" } }
]
}

```

Listing 1. The complete compiled program for “add a contact,” taken verbatim from the corpus (the AndroidWorld ContactsAddContact task). Only the schema-default null fields of each action object are omitted; everything else—state ids, verification predicates with their timeouts, per-state descriptions, and the full resource-id selectors—is reproduced exactly. Figure 3 draws the same program.

The three values typed during the trace are recorded as the parameters `first_name`, `last_name`, `phone_number` and re-bound to new values on every retrieval, so the one program generalizes across contacts. The two transitions out of `contacts_app_open` (an `action_navigate` that opens Contacts and the `action_click` on the create button) are a small redundancy the compiler emitted and the replayer tolerates; both reach `create_contact_form`.

Formally a program is a tuple $P = (S, T, M, V)$: states S (id, description, verification predicate), transitions T (each (s_i, s_j, a) firing action a to move from s_i to s_j), metadata M (task description, app context, parameter schema, and a *dedup signature* that identifies which task family the program serves), and data-extraction predicates V for question-answering tasks (e.g. `inspect_text`). The full action vocabulary is in Appendix A. This graph form buys three things a flat script cannot: **(1)** per-state verification, so the agent knows immediately when an action did not have the intended effect; **(2)** branching, since one state can have several outgoing transitions chosen by predicate (a delete-confirmation dialog that appears only on some app versions); and **(3)** in-place extension, since a new state and transition can be spliced in without regenerating the whole program.

3.2 The loop: select, replay, fall back, verify

Figure 4 shows how the harness uses such programs. For a goal T :

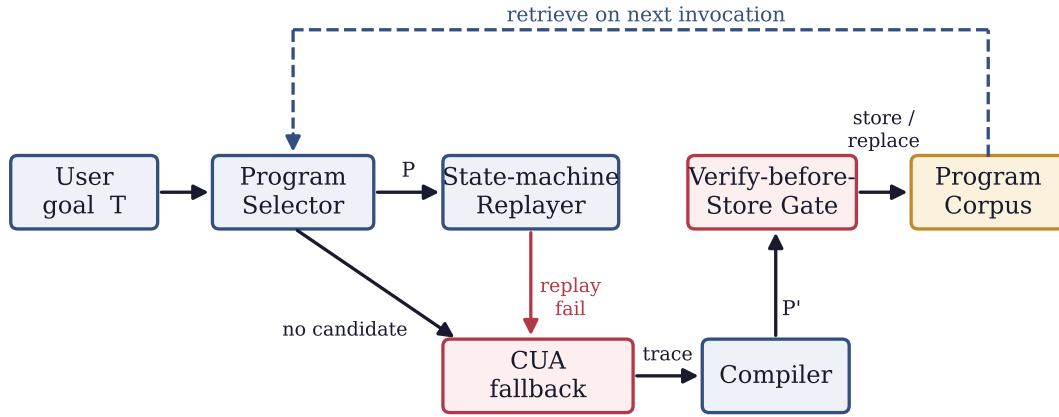


Figure 4. The PreAct harness is a verified compile–extend–replace loop. A goal is routed to the *Program Selector*; a retrieved program is run by the *Replayer*, which walks the graph and checks each predicate against the live screen. On any miss the harness falls back to the full agent (CUA), whose trace is compiled into a new program P' . P' enters the corpus only through the *Verify-before-Store Gate*. The corpus is the only growing structure; the harness code is fixed.

1. **Select.** The Program Selector queries the corpus with T and returns either a candidate program P or “no candidate.”
2. **Replay.** If P exists, the Replayer walks its graph—checking each state’s predicate against the live screen and firing each transition’s action. “Add Emilia Gonzalez” on Tuesday is solved here, in seconds, with no language-model call.
3. **Fall back.** If a predicate is false or an action errors, the harness hands control to the full agent (CUA), which continues from the live screen and records a fresh trace.
4. **Verify and store.** On success the trace is compiled into a new program P' ; the verify-before-store gate (§3.3) re-runs P' from a clean state and stores it—replacing any program with the same dedup signature—only if it independently passes.

The static harness code defines this loop; the corpus is what grows, and step 4’s gate is what lets it grow *monotonically* rather than accumulate programs that run but do not work. Algorithm 1 makes the loop precise.

Three properties of Algorithm 1 are worth highlighting. First, **the harness’s only side effect on C is line 16’s UPSERT call, gated by line 15’s double predicate**: programs enter the corpus only after independent re-validation. Second, **the corpus grows monotonically in expectation**: each UPSERT call adds a program that has independently re-passed both replay and evaluation, so in expectation the corpus quality does not decrease across UPSERT events (a stored program can still fail on a later parameter binding—§5.5—so this is an expectation, not a guarantee). Third, **the corpus mutates via UPSERT, not just append**: a fresh program with the same dedup signature replaces an older one, allowing the corpus to refine its representation of repeated tasks rather than just accumulate variants.

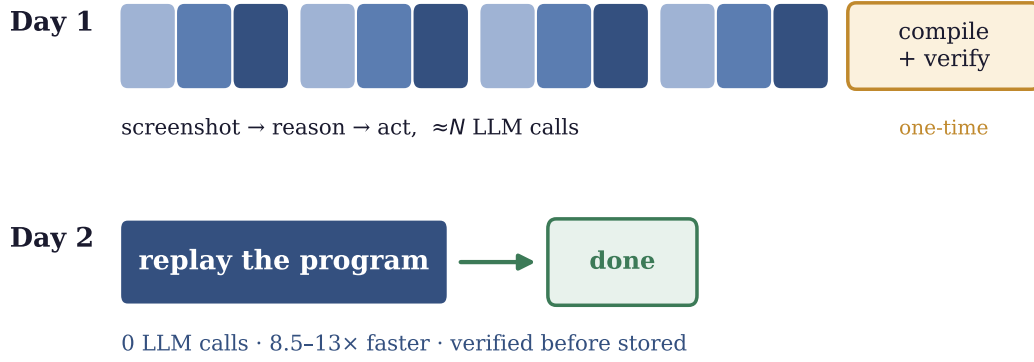


Figure 5. Where the cost goes. The first time PreAct sees a task it runs the full agent (top: $\approx N$ rounds of observe–reason–act) and pays a one-time compile-and-verify premium. Every later run replays the stored program (bottom): no language-model calls, an order of magnitude faster, and already verified before it was stored.

Algorithm 1 PreAct verified compile-extend-replace loop

Require: Task goal T , environment env , program corpus C , replay-coverage threshold τ (default 0.5)

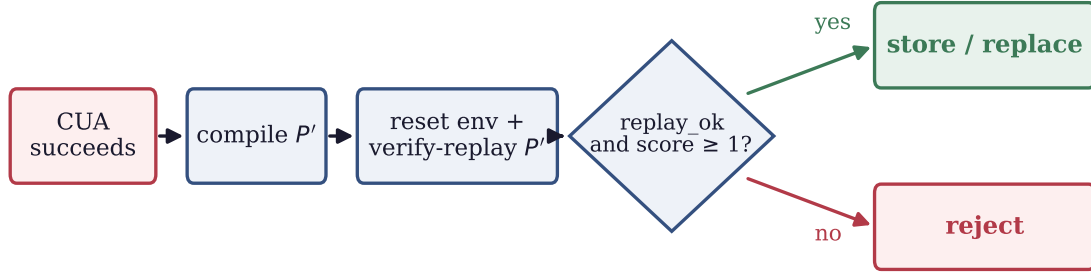
```

1:  $P \leftarrow \text{SELECTOR}(T, C)$  ▷ LLM-agentic; may return  $\perp$ 
2:  $r \leftarrow \text{NONE}$ 
3: if  $P \neq \perp$  then
4:    $r \leftarrow \text{REPLAY}(P, env)$  ▷ walk graph; verify each state predicate
5:   if  $r.success \wedge r.cov > \tau$  then
6:     return  $r$  ▷ warm path: trusted replay
7:   end if
8: end if
9:  $r \leftarrow \text{CUA}(T, env, r)$  ▷ hybrid: when  $r \neq \text{NONE}$ , CUA continues from the partial-replay state
10: if  $\neg r.success$  then return  $r$ 
11: end if
12:  $P' \leftarrow \text{COMPILE}(r.trace)$  ▷ LLM-driven trace → state-machine
13:  $env' \leftarrow \text{RESET}(env, T)$  ▷ independent re-evaluation environment
14:  $r' \leftarrow \text{REPLAY}(P', env')$ 
15:  $score' \leftarrow \text{EVALUATE}(env', T)$ 
16: if  $r'.success \wedge score' \geq 1.0$  then ▷ double gate
17:    $C \leftarrow \text{UPSERT}(C, P')$  ▷ insert by dedup signature; replace if collision
18: end if
19: return  $r$ 

```

3.3 The verify-before-store gate

The gate is the mechanism that separates PreAct from a blind cache. When the agent succeeds and the trace is compiled into a candidate P' , the harness does *not* trust it. It resets the environment to the task’s clean initial state, replays P' there, and asks the benchmark’s own



rejects the "runs but doesn't work" case: cov=100%, score=0 (the contact is never written)

Figure 6. The verify-before-store gate. A freshly compiled program is re-run from a clean state and re-scored by an independent evaluator; it enters the corpus only if it both replays cleanly *and* passes. This is what rejects the “runs but doesn’t work” programs—the ones that replay to full coverage yet leave the task unsolved.

evaluator whether the task is solved (Figure 6). P' is stored only if *both* hold: the replay ran to its terminal state without error, *and* the evaluator scores it a pass.

Both halves earn their place. Checking the evaluator’s pass alone lets through programs whose actions silently no-op (a malformed action the backend swallows) while the environment happens to still hold passing state from an earlier step—so we additionally require the replay to report its own success. The converse and more important failure mode is a clean replay that nonetheless does not pass: the program executes every action to completion (100% coverage) yet the end state does not satisfy the evaluator—the contact form was filled and “Save” was tapped, but a stale field meant no contact was written. We call this the **cov=100%/score=0 lossy replay**, and §4.3 shows it is exactly what the gate must catch.

One assumption is built into this design: because the gate verifies by *re-running* the program rather than inspecting it, the verify step performs the task’s actions a second time. This is safe in our benchmarks, where every task (and every verify-replay) starts from a scripted environment reset, but it presumes a resettable or otherwise idempotent task; tasks with irreversible side effects (sending a message, charging a card) would need a side-effect-free verification path instead. We return to this in limitation L6 (§5.6).

3.4 Selector and fallback

Two supporting components complete the loop. The **Program Selector** decides which stored program (if any) applies to a new goal; by default it is an LLM that reads the goal and each program’s description and parameter schema and chooses via a tool call, but §6 shows a plain embedding retriever does just as well—the choice of selector makes little difference to the result. The **CUA fallback** is the full agent, invoked whenever the selector finds no candidate or a replay fails; we use AndroidWorld’s T3A agent on mobile and Anthropic’s Computer-Use API on desktop and web. Notably, the production fallback runs the *verbatim* upstream agent prompt with no PreAct-specific additions (§4.6)—the harness, not the prompt, is what makes the system work.

4 Empirical Validation

4.1 Setup

Benchmarks. We evaluate on (a) AndroidWorld [36] “official-15” subset (15 tasks across 8 application domains; the curated subset used by the upstream T3A baseline), (b) OSWorld [52] “test_tiny” subset (6 tasks across Chrome, LibreOffice Calc, and LibreOffice Writer), and (c) a 12-task shopping_admin subset of WebArena [64] (string-match answer-extraction tasks on a Magento e-commerce admin panel).

Models. For Android CUA, we use Gemini 3 Flash via the multi-provider port; this is roughly 10× cheaper than Claude Sonnet 4.6 at equivalent SR (we confirm this with cross-model replication in §4.2). For OSWorld and WebArena CUA, we use Claude Sonnet 4.6 via Anthropic’s Computer-Use API. For program compilation and selector reasoning, we use Claude Sonnet 4.6 throughout.

Container. AndroidWorld runs in the public `android_world` container image, extended with a small patch that exposes the device’s screenshot and accessibility tree to the agent (Appendix C). OSWorld uses the standard `osworld-docker` image via its environment provider. WebArena uses the canonical `shopping_admin` Magento Docker stack.

Multi-seed protocol. For each seed in {42, 100, 1337, 2024, 7777}, we set the existing corpus aside (preserving prior state) and start from a fresh empty corpus. We then run cold (empty corpus) followed by warm (cold-built corpus) on the same task list.

Cost and protocol. The full validation push ran in ≈ 50 hours of autonomous container time for $\approx \$30$ – $\$35$ in LLM spend. Every claim below uses paired cold/warm runs: “cold” starts from an empty corpus; “warm” reuses the corpus the cold run built on the same tasks. This is the setting the rerun crisis lives in—the same user, the same tasks, a second time.

Metrics. We report task success rate *and* the cost of achieving it—wall-clock time, language-model token spend, and replay coverage—rather than success alone. The three benchmarks we build on score only whether a task passes, which makes an agent that re-derives every repeated task indistinguishable from one that has learned to replay it. Because PreAct’s whole premise is that repeated work should get cheaper, not just stay correct (§1), we treat this efficiency dimension as a first-class outcome: warm-run speedup and the one-time compile-and-verify overhead are measured quantities (Figures 1, 8), not illustrative ones.

A one-paragraph map of what follows. Two harness mechanisms move the success rate: the *verify-before-store gate* (§4.3) and the *cache-miss fallback* (§4.4). The runtime add-ons we tried—prompt content, runtime guardrails, and the choice of selector—do not (§4.6); the one remaining knob, the step-budget cap, trades success rate for wall-clock time by design rather than adding any. **The harness is what works.**

4.2 Repeated runs get faster

The basic promise is that the second time the agent sees a task, it is faster and no worse. Figure 7 shows the success rate rising from cold to warm on AndroidWorld official-15 across three Gemini 3 Flash seeds (corpus reset per seed): every seed improves, mean +1.33 tasks, with *no* seed regressing. The gain is not free retrieval of luck—on the warm runs the agent

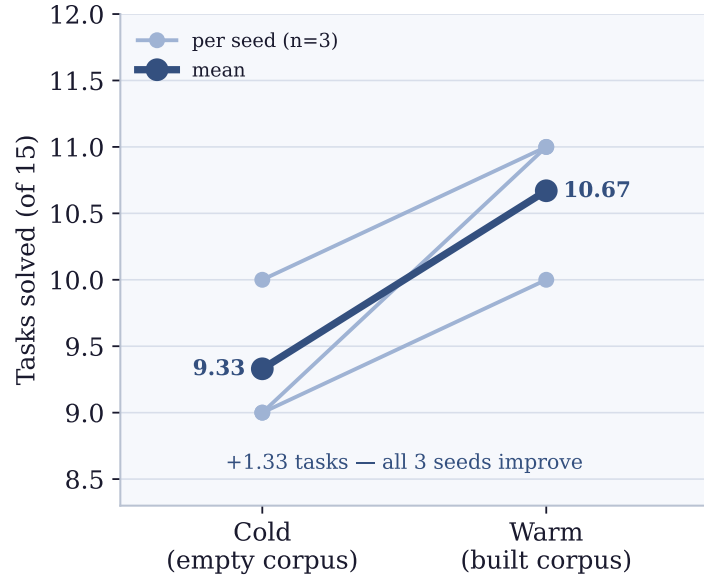


Figure 7. Cold→warm refinement is monotonic. AndroidWorld official-15 success rate, three Gemini 3 Flash seeds with the corpus reset per seed. Every seed improves (mean 9.33 → 10.67 of 15, $\Delta = +1.33$); none regresses. Per-task detail in Appendix B.

handles a majority of tasks by replaying a stored program rather than re-deriving it. Of the 13 tasks that clear setup on seed 42, 8 shift from fresh-agent solving on cold to replay-or-hybrid on warm (versus all 13 re-derived by the fresh agent on cold), which is the corpus doing its job.

The speedup is real. A replayed program carries no per-step language-model call, so warm runs are dramatically cheaper: on WebArena, served replays complete at 8.5–13 $\times$ lower cost than the corresponding fresh-agent solve (Figure 1), and on Android a warm task that hits the corpus finishes in seconds. The one-time price is the compile-and-verify step, which costs roughly as much as one more solve (+162% wall time on Android, +217% on OSWorld per stored program; Figure 8); it is repaid the first time the task recurs. Figure 5 traces where this cost goes: a one-time premium on the first solve, then near-free replays on every later run.

The result is not model-specific. Swapping Gemini 3 Flash for Claude Sonnet 4.6 on the same subset gives the same 73.3% (11/15) and—tellingly—the *same* three stable failures (BrowserDraw, SystemBrightnessMax, SystemWifiTurnOn) across both models and all three seeds. These three are properties of the benchmark’s UI (a canvas not exposed in the accessibility tree; a brightness slider that rejects scroll; a stale status-bar Wi-Fi icon), not of the agent’s reasoning—they bound the benchmark, not the method (§5.4).

4.3 The verify-gate keeps the speedup from rotting

Reuse cuts both ways. The same corpus that makes warm runs cheap can make them *wrong*, if it stores a program that runs but does not work. Recall the lossy contact program from §3.3: it taps through the form and presses Save, replays at 100% coverage, reports success—and yet writes no contact, because a stale field left the name blank. Store that program, and every future “add a contact” silently fails. The verify-before-store gate exists to keep such programs out. To show it is doing real work, we turn it off.

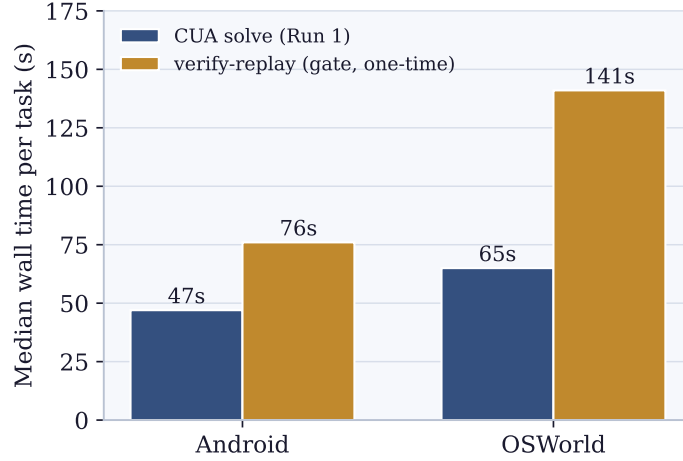


Figure 8. The gate’s one-time cost. Verifying a freshly compiled program (re-running it from a clean state) costs about as much as—or, on both platforms measured, somewhat more than—the original solve (+162% wall time on Android, +217% on OSWorld), the gap being largest on OSWorld where both the solve and the verify-replay are slowest. This is paid once per stored program and amortized over every warm run.

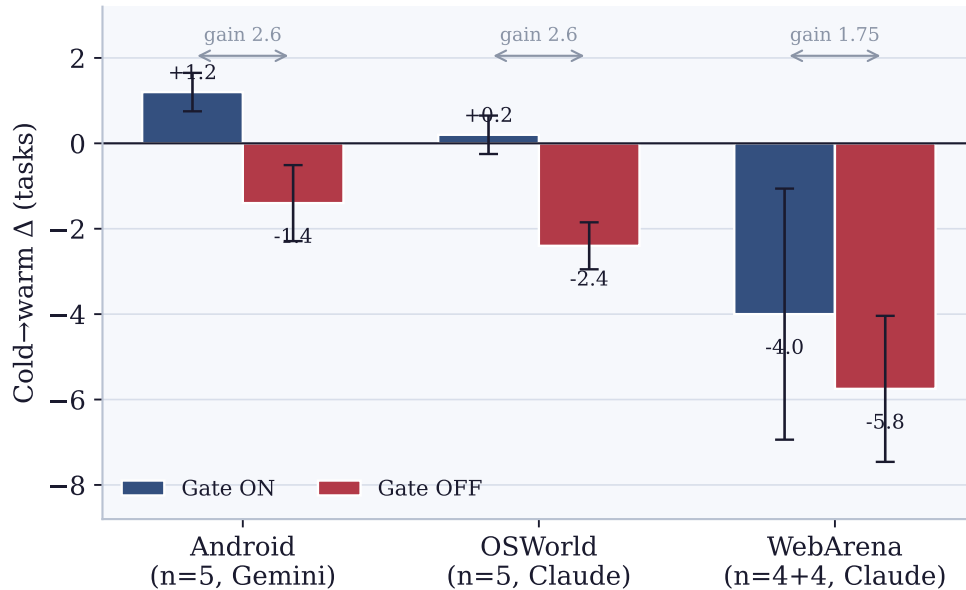


Figure 9. The gate is what makes repeated runs improve. Mean cold→warm success-rate change with the gate on (blue) versus off (red), ± 1 s.d. On all three platforms the gate-on bar is higher: the gate makes the corpus improve the agent on mobile and desktop, and degrade it far less on the web (where most programs cannot be re-verified, §4.3); without it, the corpus *degrades* the agent everywhere as lossy programs accumulate. The gap between the bars (the gate’s marginal value) is 1.75–2.6 tasks and points the same way everywhere—this is a property of the harness, not of any one platform. Per-seed and per-rep data in Appendix B.

We ran the same 2×2 ablation—cold/warm $\times$ gate on/off—on all three platforms: Android-World official-15 ($n=5$ seeds, Gemini), OSWorld test_tiny ($n=5$ reps, Claude), and a 12-task WebArena shopping_admin subset ($n=4+4$ reps, Claude). Table 2 gives the per-platform cold→warm change in each condition; Figure 9 plots it.

Table 2. Verify-gate ablation across three platforms. Each cell is the mean cold→warm change in tasks solved (± 1 s.d.). With the gate *on*, repeated runs hold or improve on mobile and desktop and regress less on the web; with it *off*, every platform regresses further as lossy programs accumulate. “Gate’s gain” is the gap between the two—the cost of not verifying. The gate-on Δ is higher than gate-off on all three platforms, strictly so in 13 of the 14 paired runs (the 14th, WebArena rep 4, is a tie). Per-seed and per-rep tables are in Appendix B.

Platform (model)	Δ gate ON	Δ gate OFF	Gate’s gain
AndroidWorld official-15 (Gemini, $n=5$)	$+1.2 \pm 0.45$	-1.4 ± 0.89	2.6
OSWorld test_tiny (Claude, $n=5$)	$+0.2 \pm 0.45$	-2.4 ± 0.55	2.6
WebArena shopping_admin (Claude, $n=4+4$)	-4.0 ± 2.94	-5.75 ± 1.71	1.75

The gate decides whether reuse helps or hurts. Read the table one row at a time. On mobile and desktop the gate-on agent improves or holds across repeated runs ($\Delta \geq 0$), while the gate-off agent *loses* ground—it does the same tasks a second time and solves fewer of them, because the corpus is now seeded with programs that replay but do not work. The effect points the same way on all three platforms and in every non-tied paired run (13 of 14; the 14th, WebArena rep 4, is a tie); per-platform sign tests give $p \approx 0.031$ on Android and OSWorld and $p = 0.125$ on WebArena’s three non-tied pairs. Pooling the consistent direction across platforms gives $p \approx 10^{-4}$ under a no-effect null; because the within-platform repetitions are correlated rather than independent, we read that pooled figure as a directional lower bound, and rest the claim on the per-platform tests and the consistent direction. That the gate’s gain lands in the same 1.75–2.6 task band despite three platforms, three models, and three evaluator styles suggests it reflects how often the *compiler* emits a lossy program—a property of compilation, not of any platform (§5.2).

Why WebArena is low even with the gate on. On WebArena the gate-on agent still regresses (-4.0), because its tasks are answer-extraction (“what was the top-selling product in 2022?”): almost every compiled program reads a value off a page that has since changed, so the gate—correctly—rejects nearly all of them, leaving the corpus empty and nothing to replay. The gate is behaving exactly as designed (it never stores a program that fails re-verification); the missing piece is what to do when it empties the corpus, which is the subject of §4.4.

The failure, observed directly. The failure the gate intercepts is directly observable. Under gate-off we logged five distinct programs (Table 3) that replay to 100% coverage, report success—and score 0 on the evaluator. The first is our running example: a stored “add a contact” program that taps through the whole form and presses Save, yet leaves no contact behind. On WebArena the same pattern is near-total: all 48 gate-off warm replays score 0. These are precisely what the gate exists to reject before storage.

4.4 The second mechanism: a fallback that closes the gap to record-and-replay

The gate exposes a question: when it empties the corpus for a task (as it does on most WebArena tasks), what should the harness do? To answer it we compared PreAct against the strongest record-and-replay baseline, **Muscle-Mem** [34]—which caches the linear action sequence, replays it blindly, and re-runs the full agent on any cache miss—and against **Workflow-Use** [4], which stores per-task scripts with no verification. Same harness, tasks, and backend

Table 3. Five reproducible “runs-but-doesn’t-work” programs logged under gate-off: each replays its full action sequence (100% coverage) yet the evaluator scores it 0. The first is the running example. With the gate on, these are rejected at compile time and never enter the corpus.

Platform	Task	What goes wrong
Android	ContactsAddContact	Replay completes; no contact written
Android	MarkorCreateFolder	Replay completes; folder not at expected path
OSWorld	Chrome history clean	Replay completes; browser state mismatches
OSWorld	LibreOffice Calc formula	Replay completes; cell formula wrong
OSWorld	LibreOffice Calc chart	Replay completes; chart properties mismatch

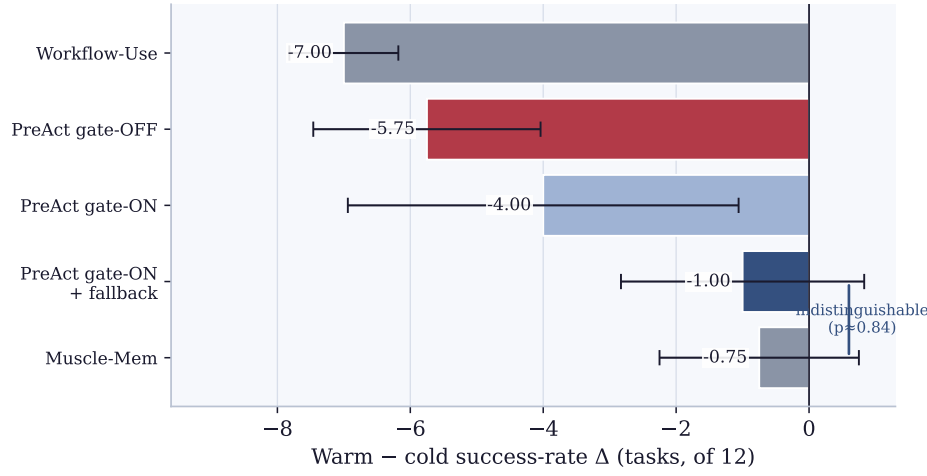


Figure 10. Closing the gap to Muscle-Mem on WebArena. Warm-cold success-rate change (± 1 s.d.). As shipped, PreAct’s gate alone (-4.0) trails Muscle-Mem (-0.75): when the gate empties the corpus, PreAct had nothing to fall back to, whereas Muscle-Mem’s cache miss re-runs the full agent. Adding the same cache-miss fallback to PreAct (-1.0) makes the two statistically indistinguishable ($p \approx 0.84$). Workflow-Use and gate-off collapse to 0 warm, for the same reason gate-off does in Figure 9.

(Claude Sonnet 4.6), $n=4$ reps each (Figure 10, Table 4).

The honest reading of Table 4 unfolds in two stages. *Stage 1 (PreAct as-is vs. baselines).* At matched $n=4$, the as-shipped **PreAct gate-ON harness underperforms Muscle-Mem on this benchmark**: Muscle-Mem’s warm-SR mean ($6.25/12 = 52\%$) is more than $3\times$ PreAct gate-ON’s ($2.0/12 = 17\%$). The warm- Δ gap is -0.75 vs. -4.0 , i.e. Muscle-Mem loses about 3.25 fewer tasks than PreAct on the cold-to-warm transition. Workflow-Use, by contrast, behaves like PreAct gate-OFF (warm SR 0/12 in every rep)—it stores compiled scripts without verification and, like gate-OFF, suffers complete warm-time collapse when the scripts encounter live-page selectors that don’t match.

Mechanism diagnosis: cache miss vs. verify-reject. WebArena’s two-run protocol replays the *same task* on Run 2, so the relevant question is what happens when the stored artifact does not match the live page. Muscle-Mem’s blind-replay attempt fails fast (typical ~ 30 s before timing out on a missing selector), and the harness then falls through to a fresh CUA run that re-solves the task with the original LLM cost. PreAct’s verify-before-store gate, by contrast, runs an independent verify-replay between Run 1 and Run 2; on these WebArena tasks the compiled state-machine predicates are brittle enough (e.g. `state=reviews_index_page`

Table 4. Head-to-head baselines on WebArena shopping_admin 12-task subset, $n=4$ reps each. PreAct (gate-ON / gate-OFF) rows reproduced from §4.3; the bottom row is the cache-miss-to-CUA fallback harness fix (§4.4.1) that we propose and measure.

System	Cold SR	Warm SR	Δ (warm–cold)	Notes
Muscle-Mem (blind linear cache)	7.0 ± 1.41	6.25 ± 0.96	-0.75 ± 1.50	Cache miss $\rightarrow$ CUA fallback per task
Workflow-Use (per-task scripts)	7.0 ± 0.82	0 ± 0	-7.0 ± 0.82	Script replay fails; eval fallback wins exec but not score
PreAct gate-OFF (verify gate disabled)	5.75 ± 1.71	0 ± 0	-5.75 ± 1.71	Stores every compile; warm SR collapses on lossy programs
PreAct gate-ON (verify gate enabled)	6.0 ± 0.82	2.0 ± 2.31	-4.0 ± 2.94	Gate rejects $\approx 83\%$ of compiles; bimodal warm SR (4/4/0/0)
PreAct gate-ON + cache-miss CUA fallback	7.25 ± 1.50	6.25 ± 0.96	-1.0 ± 1.83	On gate-reject, run a fresh CUA on Run 2 (matches Muscle-Mem semantics)

matched via XPath patterns) that they fail re-verification even on the cold-pass task, so the gate deletes the program and Run 2 has *no artifact* to fall back to. *Muscle-Mem’s advantage on WebArena is not that its replay is more reliable—it isn’t, with reps showing cache miss rates of 50–67%—but that its cache-miss path falls through cleanly to CUA, whereas PreAct’s gate-reject path leaves Run 2 with no artifact at all.*

4.4.1 Closing the Gap: Cache-Miss-to-CUA Fallback

The diagnosis suggests a small change: when the gate rejects all compiles for a task, run a fresh CUA on Run 2 instead of returning “no artifact”. We add a fallback branch that triggers a fresh CUA exploration whenever the verify-gate has emptied the corpus for the current task; we additionally tighten the gate’s cleanup to delete every program added during the task’s compile-and-verify cycle (Run-1 compile *and* any programs the verify-replay’s own CUA recovery happens to add), so a stale verify-recovery program cannot suppress the cache-miss branch. With this change, Run 2 fires a fresh CUA exactly when Muscle-Mem would: the bottom row of Table 4.

The result: **warm SR rises from 2.0/12 to 6.25/12, exactly matching Muscle-Mem’s 6.25/12, with the warm- Δ gap closing from -4.0 to -1.0 .** Across the four reps the warm-SR sequence is 6, 7, 5, 7 (vs. Muscle-Mem 5, 6, 7, 7) and the per-rep warm- Δ values are 0, +1, -3 , -2 (vs. Muscle-Mem -2 , 0, +1, -2). Welch’s two-sided t -test on the warm- Δ distributions gives $|t| \approx 0.21$, $p \approx 0.84$: under any conventional threshold we cannot reject equality of means, i.e. PreAct-with-fallback and Muscle-Mem are statistically indistinguishable on this benchmark. The cold-SR comparison is similar (7.25 vs. 7.0, $p > 0.5$).

This is a small-but-real *positive* finding for the architecture: PreAct’s gate-rejection signal is high quality (the rejections in the gate-ON-no-fallback condition were correct—the underlying compiled programs really were lossy), and the only thing missing was a fallback path. Critically, the fallback does *not* compromise the gate’s Android/OSWorld value: the gate continues to reject lossy compiles *before* they enter the corpus, preserving monotonicity (§4.3); the fallback

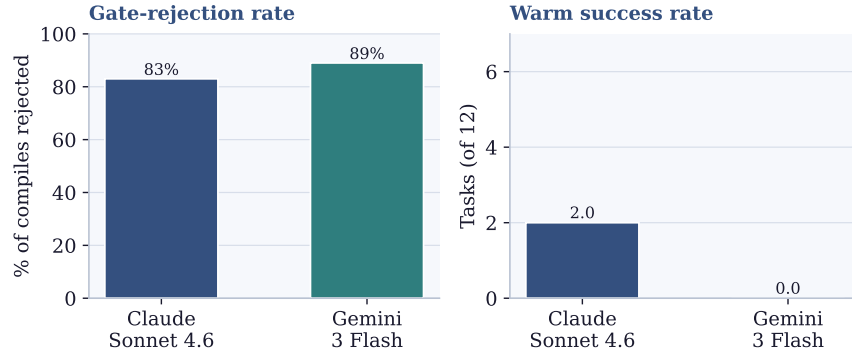


Figure 11. The gate is mechanism-driven, not Claude-specific. Swapping the compile-step model from Claude to Gemini 3 Flash leaves the gate-rejection rate in the same band (83% vs. 89%); warm success rate depends only on what survives the gate.

Table 5. Compile-LLM robustness: gate behavior under Claude vs Gemini compile, WebArena 12-task shopping_admin gate-ON (no cache-miss fallback).

Compile LLM	Cold SR	Warm SR	Δ	Gate-rejection rate
Claude Sonnet 4.6 ($n=4$ reps)	$6.0/12 \pm 0.82$	$2.0/12 \pm 2.31$	-4.0 ± 2.94	$\approx 83\%$
Gemini 3 Flash ($n=3$ reps)	$6.33/12 \pm 0.58$	$0/12 \pm 0$	-6.33 ± 0.58	$\approx 89\%$

only changes what happens on the cache-miss *branch* of Run 2 on the platform where the gate’s rejection rate is highest. The fallback also does not save tokens compared to Muscle-Mem on this benchmark—both pay the full CUA cost when the cache misses, so warm tokens are similar to cold (~ 65 k for fallback vs. ~ 50 k cold; the slight increase comes from CUA re-exploring with a fresh browser session)—but it recovers the warm-SR property that the as-shipped harness was leaving on the table.

Implication. Across the three platforms tested, the harness needed two cooperating mechanisms: the verify-before-store gate to keep the corpus monotonic (§4.3; Android +2.6, OSWorld +2.6, WebArena +1.75 task diff-of-deltas under the gate), *and* a cache-miss-to-CUA fallback when the gate empties the corpus on a particular task (this section; WebArena $\Delta = -1.0$ matching Muscle-Mem’s -0.75). The two-stage finding—“gate alone” loses to Muscle-Mem on WebArena, but “gate + cache-miss fallback” matches Muscle-Mem on WebArena *and* retains the gate’s Android/OSWorld monotonicity—is the main architectural lesson of this section. Pre-Act enables this cache-miss fallback by default; disabling it recovers the gate-ON, no-fallback behavior reported as the $\Delta = -4.0$ measurement.

4.5 Compile-LLM Robustness: Gemini vs Claude Compile

To probe whether the verify-gate’s compile-rejection rate is Claude-specific or a property of the compile step itself, we swap the compile-step LLM from Claude Sonnet 4.6 to Gemini 3 Flash, leaving the CUA loop and selector on Claude. We ran $n=3$ WebArena reps on the same 12-task shopping_admin subset (gate-ON, no cache-miss fallback, for clean comparison with the Claude-compile baseline in §4.3); Table 5 and Figure 11 report the result.

Cold SR is statistically indistinguishable across compilers (6.0 vs. 6.33, $p > 0.5$), as expected

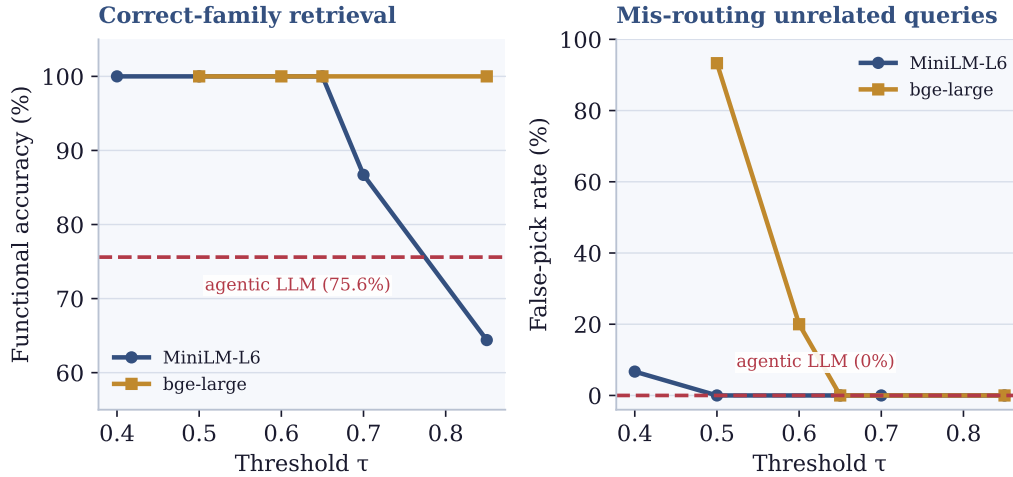


Figure 12. A plain embedding retriever matches the LLM-based selector. On the 58-program corpus, both embedding backbones reach 100% correct-family retrieval (left) at 0% mis-routing of unrelated queries (right) once the similarity threshold τ is tuned, against the agentic selector’s 75.6% (dashed). The choice of selector thus barely affects retrieval quality—a tuned embedding retriever even edges out the agentic one, and the agentic selector’s own misses are conservative no-picks that fall back to CUA rather than wrong retrievals; we keep it as the default only for its interpretable logs and tuning-free operation. Full sweep in Appendix F.

since both use the same Claude Sonnet 4.6 CUA loop on Run 1. The gate’s rejection rate is in the same band (83% with Claude vs. 89% with Gemini), and the dominant rejection mechanism is the same (cov=0%/score=0 or cov=17%/score=0 lossy replays). Qualitatively, Gemini compiles tend to fail at very-first-step replay (cov=0% modal) whereas Claude compiles run to completion but produce wrong answers (cov=100% modal); both failure modes are caught by the gate. The Gemini-compile warm SR is 0/12 on all three reps with $\sigma=0$ —tighter than the Claude bimodal distribution (4, 4, 0, 0)—reflecting Gemini’s higher rate of cov=0% compiles that the gate rejects with 100% consistency. **The verify-gate is mechanism-driven, not Claude-specific:** substituting Gemini 3 Flash at the compile step does not change the qualitative behavior (gate fires; lossy compiles get rejected; warm SR depends on what survives), and the quantitative rejection rate shifts only modestly (83 $\rightarrow$ 89%). Cross-model replication closes the compile-step concern noted under L1 in Appendix E.

4.6 What Did Not Change the Result

The AB experiments that came back *negative* or marginal are just as useful: they rule out the obvious alternative explanations for why PreAct works. Detailed per-seed tables are in Appendix F.

The choice of program selector barely matters. We had assumed an LLM-based selector was necessary—deciding whether a stored program applies to a new task looked like a reasoning problem. It is not: a plain embedding retriever (MiniLM-L6 or bge-large) matches or beats the agentic selector on the 58-program corpus, reaching 100% correct-family retrieval at 0% mis-routing once its threshold is tuned, versus the agentic selector’s 75.6% (Figure 12). We keep the agentic selector as the default only for its interpretable reasoning logs and because it needs no threshold tuning—not because it retrieves better.

Prompt-level PreAct content is unused in production. Our implementation includes three optional PreAct-specific guideline bullets, but the production CUA path never passes them to the agent. The 73.3% Android SR is achieved with the verbatim upstream T3A prompt and zero PreAct prompt-level additions. The contribution on Android is the harness wrapping T3A—not the prompt.

Code-level runtime guardrails are aggregate-neutral at $n=5$. A 2×2 ablation of the guardrails (a double-tap before text entry, an image-task no-op cap, and scroll-exhaustion notes) gives *identical* cold means ($10.2 = 10.2$) and warm means 11.0 (on) vs. 10.6 (off), a 0.4-task gap well within seed-level variance (Figure 15; Table 13). Per-task analysis shows the double-tap helps the audio-record-with-filename task (a populated filename dialog) but the gain is cancelled by minor timing penalties on Camera tasks.

State-machine vs. flat-script runtime: small but consistent advantage. A runtime-mode ablation (state-machine vs. flat-script; Figure 16 and Table 14 in Appendix F) gives flat-script 10.6 ± 1.14 ($n=5$ seeds) vs. state-machine 11.67 ± 0.58 on the 3 matched seeds; matched-seed $\Delta = +0.67$ tasks favoring state-machine, all 3 paired comparisons in the non-regressive direction (sign-test $p = 0.125$, suggestive). The wins concentrate on Camera tasks where verification catches mismatches linear execution misses. The architectural commitment matters but is a smaller effect than the verify-gate’s 1.75–2.6-task diff-of-deltas.

Dynamic step-budget cap: a deliberate speed/accuracy tradeoff. Unlike the add-ons above, the step-budget cap is *not* neutral—it is the one knob in this section that moves the success rate, and it moves it by choice rather than as a side effect of the harness. The cap—a budget of $8 \times$ the task’s step-complexity, clamped between 20 and 30 steps—vs. unbounded 60-step at $n=5$ gives 9.8 ± 0.84 (capped) vs. 11.6 ± 0.55 (unbounded): the cap *costs* $\Delta = 1.8 \pm 0.84$ tasks of success rate (within the ± 2 pre-registered prediction interval) in exchange for $\approx 30\%$ less wall time on the FAIL tail (the unbounded budget burns the extra time largely on SystemBrightnessMax, which is harness-deterministic and never solved either way). We ship the capped setting as the latency-favoring default; deployments that prioritize success rate over latency should raise or remove the cap. We list it here because it is the natural place to compare it against the genuinely-neutral add-ons, not because it is itself neutral.

Per-seed tables for these negative ablations are in Appendix F, and a complete map of which validity threat each experiment closes—twelve in all, eleven in scope and closed at the rigor noted per threat—is in Appendix H.

5 Discussion

5.1 Why the Gate Matters

The mechanism is: the verify-gate filters lossy compiles before they enter the corpus. “Lossy” here means: action sequences that are mechanically faithful (cov=100% on replay) but do not produce the live evaluator’s required end state. Without the gate, every CUA-success-then-compile cycle adds a new program to the corpus regardless of whether that program actually achieves the goal under deterministic re-execution. With the gate, only programs that have independently re-passed enter the corpus.

This matters because the corpus is the input to subsequent retrieval. A retrieved lossy program will be replayed on a future invocation; the replay will execute mechanically ($\text{cov}=100\%$) and the live evaluator will score 0. Worse, the harness’s selector will continue to retrieve the lossy program until something explicitly removes it. The verify-gate prevents the corpus from accumulating these failures, which would otherwise reproduce the same wrong outcome on every future retrieval.

5.2 The Verify-Gate’s Marginal Value is Structural

A notable result is that the verify-gate’s marginal value (Figure 9) falls in the *same narrow band*—1.75 to 2.6 tasks of diff-of-deltas—on three structurally different platforms with different LLMs, different task subsets, and different evaluator semantics: 2.6 tasks on Android (Gemini, UI-predicate evaluators), 2.6 tasks on OSWorld (Claude, desktop-state evaluators), and 1.75 tasks on WebArena (Claude, string-match evaluators on a 12-task shopping_admin subset). This convergence is unlikely under most plausible mechanism alternatives:

- If the gate’s value derived from a model-specific behavior (e.g., Gemini-particular flaws), Android’s diff-of-deltas would not match OSWorld’s or WebArena’s (both Claude).
- If it derived from task-specific properties (e.g., Markor-edit ambiguity), the desktop-task and web-task diff-of-deltas would differ from the mobile case.
- If it derived from benchmark-evaluator quirks (e.g., Android’s specific scoring rubric), OSWorld’s desktop-state evaluators and WebArena’s string-match evaluators would not produce the same magnitude.

Instead, the diff-of-deltas magnitude is structural: it is the rate at which the underlying *compile* step produces lossy state-machine programs. Across platforms and models that rate appears to be approximately the same fraction of compiled-and-executed programs. The *absolute* regression varies more (43 pp on OSWorld’s 6-task subset, 17 pp on Android’s 15-task subset, 15 pp on WebArena’s 12-task subset), but the *absolute number of tasks* affected falls in the same range. We interpret this as: the lossy-compile rate is a property of the compile step (LLM trace $\rightarrow$ state-machine), not of the platform or evaluator. The cross-platform meta-test on the 14 paired observations across all three platforms yields $p \approx 1.2 \times 10^{-4}$ under the null hypothesis of no gate effect (a directional lower bound that treats within-platform reps as independent; see the correlation caveat in §4.3).

5.3 Per-Program Reproducibility of $\text{cov}=100\%/\text{score}=0$

The five $\text{cov}=100\%/\text{score}=0$ cases (Table 3) are not equally reproducible across reps—the OSWorld Calc-formula program reproduces 5/5 reps, the Calc-chart 4/5, and the Chrome-history 2/5 (per-program breakdown in Appendix G). **The aggregate regression, however, is consistent across every rep.** All 5 OFF reps lose at least 2 tasks; no rep escapes the regression. The selector and replayer’s per-task behavior is partially nondeterministic, but the gate-OFF condition reliably produces ≥ 2 -task regression on every rep. The paper’s claims rest on this aggregate-level determinism, not on per-program reproducibility.

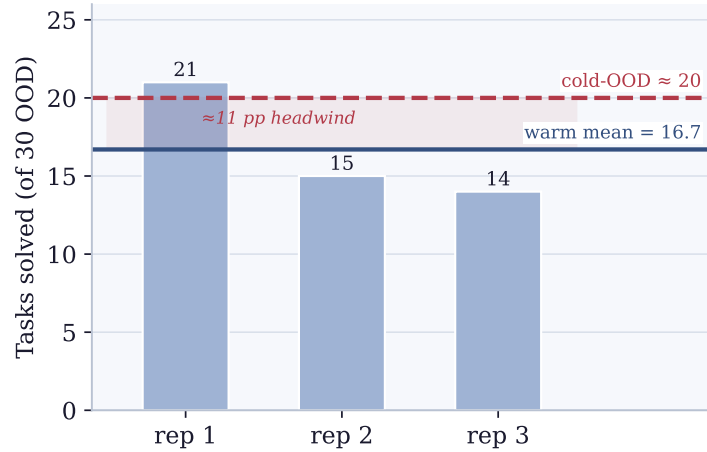


Figure 13. The corpus does not transfer cleanly. A corpus built on six OSWorld tasks, applied to 30 unseen tasks, lands ≈ 11 points *below* the cold baseline: a few mis-routed retrievals fire brittle predicates and burn replay budget. The corpus’s value is concentrated in repeated invocations of seen tasks.

5.4 Harness-Deterministic vs. LLM-Driven Failures

Three AndroidWorld tasks (BrowserDraw, SystemBrightnessMax, SystemWifiTurnOn) and one OSWorld task (LibreOffice Writer macro) fail across all five seeds, both LLM backends (Claude and Gemini), all four guardrail/budget configurations, and both verify-gate conditions. These failures are *harness-deterministic*: they are properties of the UI patterns themselves (status-bar stale reads, scroll-on-seekbar incompatibility, image-canvas-not-in-a11y-tree) rather than properties of the agent’s reasoning. They bound the benchmark, not the method.

5.5 Out-of-Distribution Generalization

We tested whether the corpus transfers beyond seen tasks by running a test_tiny-built corpus on the 33 non-test_tiny tasks of OSWorld test_small at $n=3$ reps (with the corpus copy reset between reps). Per-rep SR over the 30 OOD tasks that complete setup: 21/30, 15/30, 14/30, giving a warm-OOD mean of $16.7/30 = 55.6\% \pm 12\%$. The cold-OOD baseline on the same task subset is a single ($n=1$) pass at $\approx 20/30 = 67\%$; because it is unpaired and not multi-rep, the gap below should be read as indicative rather than a tightly-estimated effect. **The corpus does not transfer; at $n=3$ the warm-OOD mean is ≈ 11 percentage points below this cold-OOD pass** (Figure 13), a small-but-real headwind rather than the neutral outcome a single pass had suggested. Inspection of the rep logs shows the selector occasionally retrieves a near-miss program (a Chrome task’s program retrieved for a Thunderbird task whose description shares the words “open” and “message”), and the brittle XPath/state predicates from the source domain fail on the OOD page, costing replay budget without recovering via CUA fast enough. The selector’s no-pick rate on OOD queries does buffer this somewhat—it returns “no candidate” on the majority of cross-family queries—but the queries it does pick are penalised. This sharpens the scope of the “self-extending corpus” claim: **the corpus’s value is concentrated in repeated invocations of seen tasks, and corpora built on small task subsets actively work against out-of-distribution generalization on this evaluator.** Full per-domain breakdown in Appendix D.

5.6 Limitations

Beyond the eleven in-scope validity threats closed in Table 16, we explicitly acknowledge six remaining limitations:

- **(L1) Architectural baseline (partial).** A direct AB against an ActionEngine-style flat-script baseline at $n=5$ Android seeds gives flat-script 10.6 ± 1.14 vs. state-machine 11.67 ± 0.58 ($\Delta = +0.67$, $p = 0.125$): suggestive but not significant.
- **(L2) Benchmark coverage.** The 6-task OSWorld test_tiny, 15-task AndroidWorld official-15, and 12-task WebArena shopping_admin subsets are a small sample of computer-using tasks; absolute pp magnitudes are denominator-dependent (43/17/15 pp for 1.75–2.6 absolute tasks).
- **(L3) Compile cost.** Verify-replay adds median +217% wall overhead on OSWorld and +162% on Android per successful CUA-then-compile cycle; the cost is amortized on warm runs but matters for high-throughput deployments.
- **(L4) Compile-fidelity rate.** The LLM compiler produces lossy state-machines on a non-trivial fraction of inputs (audit: $\sim 28\%$ Android programs miss `navigate_back`; 100% of audited WebArena programs use `inspect_screenshot` on dynamic state); the gate filters this at storage but scales as the number of compile attempts, not stored programs.
- **(L5) Selector nondeterminism.** Verbatim retrieval is 73% exact-id (all 8 misses were semantically-equivalent picks); paraphrase robustness is 75.6% functional (47% exact-id + 29% equivalent), 24% no-pick, 0% **wrong-task-family**—the selector is conservative-by-design rather than over-eager.
- **(L6) Verification assumes a resettable or idempotent environment.** The verify-before-store gate confirms a freshly compiled program by *re-executing* it from a reset environment and consulting the task’s evaluator, and the warm replay path likewise re-runs a stored program end-to-end. Both presuppose that performing the program again is safe. Our benchmarks provide this for free—each task starts from a scripted reset—but many real tasks are not idempotent: re-running “add contact Emilia Gonzalez,” “send the email,” or “submit the payment” duplicates the side effect rather than re-confirming the goal. Outside a benchmark, the gate would need a side-effect-free way to verify (a read-only check of the end state, a sandboxed dry-run, or a transactional rollback) in place of literal re-execution. We validate the gate as a benchmark mechanism; making it safe under irreversible side effects is future work.

Detailed measurements and analysis for L1–L5 in Appendix E.

5.7 Future Work

Four directions stand out.

Scaling the corpus. The current corpus (58 Android programs after multi-week experiments) is small. We have not characterized the corpus’s behavior at 10^3 or 10^4 programs: does the agentic selector still discriminate accurately when the candidate set is large? Does dedup-signature collision become an issue? Does the verify-replay step cost become prohibitive at scale?

Architectural baselines. The state-machine-as-executable claim should be tested directly

against a flat-script baseline (e.g., an ActionEngine-style implementation that runs the same compile cycle but generates flat Python at the verify step). The current paper supports the claim by working implementation across two platforms but not by direct comparison.

Long-horizon tasks. Both AndroidWorld and OSWorld have task horizons of order 10–30 actions. Tasks requiring long-term planning, goal decomposition, or recovery from compound failures (e.g., entire workflow tasks lasting hundreds of actions) are not represented in our benchmarks. We expect the harness’s CUA-fallback mechanism to be tested most severely on such tasks; integration with explicit goal-decomposition planners is the natural next step.

Internalizing the corpus. PreAct externalizes task dynamics into inspectable programs; the converse is open—whether a model can be trained on its own verified corpus to *internalize* these programs, folding the externalized world model back into the weights while keeping the verify-gate as the signal that decides what is worth internalizing. This would trade PreAct’s inspectability for the generality an internalized world model affords, and the verified corpus is a natural source of supervision for it.

6 Conclusion

PreAct started from a simple observation: a computer-using agent should not re-derive a task it has already solved. Acting on it took two commitments. The first is about representation—what the agent remembers is a state-machine program it can run directly, not a transcript it has to re-interpret. The second is about discipline—the agent never trusts a remembered program until it has re-run it from a clean state and confirmed that the program still reaches the goal. Across three different platforms, these two ideas together let an agent get faster on repeated work while keeping its growing memory honest. The pieces we had expected to matter mostly did not: the runtime guardrails left the aggregate numbers unchanged, the prompt-level guidance was never exercised in production, and a small embedding retriever did as well as the agentic LLM selector. What carried the result was the pairing itself—an executable memory together with the refusal to keep anything unverified.

That refusal is the easy part to skip and the part that matters most. A memory that grows without verification does not make an agent better over time; it makes the agent repeat the same mistake on every reuse, retrieving a program that looks right, no longer works, and runs to its last step with nothing to show for it. As agents build up larger and longer-lived libraries of skills, the hard question stops being whether an agent can remember what it did and becomes whether it can tell which of its memories are still true. “Add Emilia Gonzalez” should take seconds the second time—but only if the agent first checks that the contact is really there.

Acknowledgements

We thank Yurun Jin and Junlong Ye for early implementations and experiments of this work. Pine Copilot, Claude Code, and Claude Opus 4.8 were used during this research.

References

- [1] Anthropic. Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku. <https://www.anthropic.com/news/3-5-models-and-computer-use>, October 2024.
- [2] Hao Bai, Yifei Zhou, Mert Cemri, Jiayi Pan, Alane Suhr, Sergey Levine, and Aviral Kumar. DigiRL: Training in-the-wild device-control agents with autonomous reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [3] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Bucker, Lawrence Jang, and Zack Hui. Windows agent arena: Evaluating multi-modal os agents at scale. *arXiv preprint arXiv:2409.08264*, 2024.
- [4] Browser-Use. Workflow-Use: Create and run workflows (RPA 2.0). <https://github.com/browser-use/workflow-use>, 2025.
- [5] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers. *arXiv preprint arXiv:2305.17126*, 2023.
- [6] Hyunjoo Chae, Namyoun Kim, Kai Tzu-iunn Ong, Minju Gwak, Gwanwoo Song, Jihoon Kim, Sunghwan Kim, Dongha Lee, and Jinyoung Yeo. Web agents with world models: Learning and leveraging environment dynamics in web navigation. *arXiv preprint arXiv:2410.13232*, 2024.
- [7] Wenhui Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research (TMLR)*, 2023. arXiv:2211.12588.
- [8] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing gui grounding for advanced visual gui agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [9] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025. ECAI 2025.
- [10] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [11] Erhu Feng, Wenbo Zhou, Zibin Liu, Le Chen, Yunpeng Dong, Cheng Zhang, Yisheng Zhao, Dong Du, Zhichao Hua, Yubin Xia, and Haibo Chen. Get experience from practice: LLM agents with record & replay. *arXiv preprint arXiv:2505.17716*, 2025.
- [12] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. arXiv:2211.10435.

- [13] Izzeddin Gur, Hiroki Furuta, Austin Huang, Mustafa Safdari, Yutaka Matsuo, Douglas Eck, and Aleksandra Faust. A real-world webagent with planning, long context understanding, and program synthesis. In *International Conference on Learning Representations (ICLR)*, 2024.
- [14] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.
- [15] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- [16] Shibo Hao, Yi Gu, Haodi Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. Reasoning with language model is planning with world model. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [17] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. Webvoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [18] Wenyi Hong, Wei Han Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxuan Dong, Ming Ding, and Jie Tang. Cogagent: A visual language model for gui agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [19] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [20] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [21] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2005.11401.
- [22] Bojie Li. User as code: Executable memory for personalized agents. *arXiv preprint arXiv:2606.16707*, 2026.
- [23] Chengshu Li, Jacky Liang, Andy Zeng, Xinyun Chen, Karol Hausman, Dorsa Sadigh, Sergey Levine, Li Fei-Fei, Fei Xia, and Brian Ichter. Chain of code: Reasoning with a language model-augmented code emulator. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2312.04474.
- [24] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023. arXiv:2209.07753.

- [25] Xiangyuan Liang, Bohan Cao, Shengyu Gu, Yifei Li, et al. SAGE: Self-evolving agents with reflective and memory-augmented abilities. *arXiv preprint arXiv:2409.00872*, 2024.
- [26] Yaxi Lu, Shenzhi Yang, Cheng Qian, Guirong Chen, Qinyu Luo, Yesai Wu, Huadong Wang, Xin Cong, Zhong Zhang, Yankai Lin, Weiwen Liu, Yasheng Wang, Zhiyuan Liu, Fangming Liu, and Maosong Sun. Proactive agent: Shifting llm agents from reactive responses to active assistance. *arXiv preprint arXiv:2410.12361*, 2024.
- [27] Aman Madaan, Shuyan Zhou, Uri Alon, Yiming Yang, and Graham Neubig. Language models of code are few-shot commonsense learners. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2022. *arXiv:2210.07128*.
- [28] Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. Skill-pro: Learning reusable skills from experience via non-parametric ppo for llm agents. *arXiv preprint arXiv:2602.01869*, 2026. ICML 2026.
- [29] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A benchmark for general ai assistants. In *International Conference on Learning Representations (ICLR)*, 2024.
- [30] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. WebGPT: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021.
- [31] Runliang Niu, Jindong Li, Shiqi Wang, Yali Fu, Xiyu Hu, Xueyuan Leng, He Kong, Yi Chang, and Qi Wang. Screenagent: A vision language model-driven computer control agent. In *Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI)*, 2024.
- [32] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [33] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.
- [34] Pig.dev. Muscle-Mem: A cache for AI agents to learn and replay complex behaviors. <https://github.com/pig-dot-dev/muscle-mem>, 2025.
- [35] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. UI-TARS: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025.
- [36] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, et al. Android-World: A dynamic benchmarking environment for autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2025.

- [37] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. Android in the wild: A large-scale dataset for android device control. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [38] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [39] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.11366.
- [40] Significant Gravitas. Auto-gpt: An autonomous gpt-4 experiment. <https://github.com/Significant-Gravitas/AutoGPT>, 2023.
- [41] Theodore R Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research (TMLR)*, 2024.
- [42] Richard Sutton. The bitter lesson. <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>, 2019. Blog post, March 13, 2019.
- [43] Geert Trooskens, Aaron Karlsberg, Anmol Sharma, Lamara De Brouwer, Max Van Puyvelde, Matthew Young, John Thickstun, and Gil Alterovitz. Compiled AI: Deterministic code generation for LLM-based workflow automation. *arXiv preprint arXiv:2604.05150*, 2026.
- [44] Wil M P van der Aalst, Martin Bichler, and Armin Heinzl. Robotic process automation. *Business & Information Systems Engineering*, 60(4):269–272, 2018.
- [45] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research (TMLR)*, 2024. arXiv:2305.16291.
- [46] Junyang Wang, Haiyang Xu, Jiabo Ye, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. Mobile-agent: Autonomous multi-modal mobile device agent with visual perception. *arXiv preprint arXiv:2401.16158*, 2024.
- [47] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better llm agents. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.01030.
- [48] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [49] Zhiruo Wang, Daniel Fried, and Graham Neubig. Trove: Inducing verifiable and efficient toolboxes for solving programmatic tasks. *arXiv preprint arXiv:2401.12869*, 2024.

- [50] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [51] Zhiyong Wu, Chengcheng Han, Zichen Ding, Zhenmin Weng, Zhoumianze Liu, Shunyu Yao, Tao Yu, and Lingpeng Kong. Os-copilot: Towards generalist computer agents with self-improvement. *arXiv preprint arXiv:2402.07456*, 2024.
- [52] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024.
- [53] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*, 2025. NeurIPS 2025.
- [54] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [55] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [56] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [57] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [58] Chaoyun Zhang, Liqun Li, Shilin He, Xu Zhang, Bo Qiao, Si Qin, Minghua Ma, Yu Kang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. UFO: A ui-focused agent for windows os interaction. *arXiv preprint arXiv:2402.07939*, 2024.
- [59] Chi Zhang, Zhao Yang, Jiaxuan Liu, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. Appagent: Multimodal agents as smartphone users. *arXiv preprint arXiv:2312.13771*, 2023.
- [60] Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents. *arXiv preprint arXiv:2404.13501*, 2024.
- [61] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.
- [62] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. Gpt-4v(ision) is a generalist web agent, if grounded. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

- [63] Hongbin Zhong, Fazle Faisal, Luis França, Tanakorn Leesatapornwongsa, Adriana Szekeres, Kexin Rong, and Suman Nath. ActionEngine: From reactive to programmatic GUI agents via state machine memory. *arXiv preprint arXiv:2602.20502*, 2026.
- [64] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2024.

Appendix A. Computer-Action Schema

Each transition’s action field must be one of the action types in Table 6. Action types are translated to platform-specific backends at execution time: `action_click` on Android maps to a JSONAction-formatted touch with the resolved element’s (x, y) coordinates; on OSWorld it maps to `pyautogui.click(x, y)`. The cross-platform schema enables the same compiled state-machine program to run on either backend, although in practice each task’s compile is platform-specific because the verification predicates use platform-specific selector dialects (Android XPath vs. pyautogui screenshot regions).

Table 6. Computer action types in PreAct’s program schema. Each transition uses exactly one type.

Action type	Description and required fields
<code>action_click</code>	Click on a UI element. <code>target</code> : selector.
<code>action_long_press</code>	Long-press on a UI element. <code>target</code> : selector.
<code>input_text</code>	Type text into a focused element. <code>text</code> : literal or \$param. <code>index</code> : optional element id.
<code>action_keypress</code>	Send a key event. <code>key</code> : Enter, Back, Tab, etc.
<code>scroll</code>	Scroll a region. <code>direction</code> : up/down/left/right. Optional <code>index</code> for sub-element scroll.
<code>open_app</code>	Launch an application by name. <code>app_name</code> : string.
<code>wait</code>	Sleep for the platform’s default delay.
<code>navigate_back</code>	Send the system Back gesture (Android) or browser-back equivalent.
<code>navigate_home</code>	Return to the home screen / desktop.
<code>inspect_text</code>	Read the value at a selector. <code>store_result_as</code> : variable name. Used by QA-style tasks.
<code>answer</code>	Emit a final textual answer. <code>text</code> : literal or computed.
<code>status</code>	Terminate the program. <code>goal_status</code> : complete/infeasible.

The description field on each transition (one short sentence) is consumed by the agentic Program Selector (§3) when judging whether a stored program applies to the current task. Selectors are written to be parameter-aware (e.g. “type the filename (parameter filename)” rather than “type text”), since the program’s parameters list is what the LLM uses to bind the program’s slots to the new task’s instance values at retrieval time.

Appendix B. Per-Seed and Per-Task Multi-Seed Data

This appendix gives the per-seed and per-rep detail behind the aggregate figures in §4.2 (Figure 7) and §4.3 (Table 2, Figure 9). The verify-gate ablation is broken out per platform in Table 8 (Android), Table 9 (OSWorld), and Table 10 (WebArena).

Table 7. Cold→warm monotonicity, AndroidWorld official-15, $n=3$ Gemini seeds with corpus reset per seed (aggregated in Figure 7). All three seeds refine monotonically. “Mode-shift” counts tasks that move from fresh-agent solving (cua) on cold to direct replay (rpa) or replay-then-fallback (hybrid) on warm, out of the tasks that clear setup.

Seed	Cold SR	Warm SR	Δ	Mode-shift on warm
42	10/15 (66.7%)	11/15 (73.3%)	+1	8/13 cua→rpa/hybrid
100	9/15 (60.0%)	11/15 (73.3%)	+2	8/11 cua→rpa/hybrid
1337	9/15 (60.0%)	10/15 (66.7%)	+1	5/10 cua→rpa/hybrid
Mean	9.33 ± 0.58	10.67 ± 0.58	+1.33 (+8.9 pp)	All 3 monotonic

Table 8. Verify-gate ablation, AndroidWorld official-15, $n=5$ seeds with corpus reset per seed (Android row of Table 2). All five gate-ON pairs are monotonic; all five gate-OFF pairs regress (zero inversions; paired sign test $p \approx 0.031$).

Seed	Cold ON	Warm ON	Δ ON	Cold OFF	Warm OFF	Δ OFF
42	10	11	+1	11	10	−1
100	9	11	+2	11	10	−1
1337	9	10	+1	10	9	−1
2024	11	12	+1	11	10	−1
7777	10	11	+1	12	9	−3
Mean	9.8	11.0	+1.2 ± 0.45	11.0	9.6	−1.4 ± 0.89

Table 9. Verify-gate ablation, OSWorld test_tiny, $n=5$ reps, Claude Sonnet 4.6 (OSWorld row of Table 2). All five gate-OFF reps regress by ≥ 2 tasks; paired sign test $p \approx 0.031$.

Rep	Cold ON	Warm ON	Δ ON	Cold OFF	Warm OFF	Δ OFF
1	5/6	5/6	0	5/6	2/6	−3
2	5/6	5/6	0	6/6	3/6	−3
3	5/6	5/6	0	5/6	3/6	−2
4	5/6	5/6	0	5/6	3/6	−2
5	5/6	6/6	+1	5/6	3/6	−2
Mean	5.0	5.2	+0.2 ± 0.45	5.2	2.8	−2.4 ± 0.55

The gate-on warm SR is bimodal across reps (4, 4, 0, 0): on reps 1–2 a compiled program survives the gate and serves a partial replay, whereas on reps 3–4 the gate empties the corpus and, with no cache-miss fallback in this condition, warm SR drops to 0. This is the WebArena-specific pattern §4.4 resolves by adding the cache-miss-to-CUA fallback. Even so, gate-on loses fewer tasks than gate-off in every non-tied rep, matching the cross-platform direction.

Table 10. Verify-gate ablation, WebArena shopping_admin 12-task subset, $n=4+4$ reps, Claude Sonnet 4.6 CUA + Claude compile, gate-ON without the cache-miss fallback (WebArena row of Table 2; SR out of 12 tasks). Every gate-OFF warm run collapses to 0 (all 48 warm replays score 0); the gate-on column loses fewer tasks on every non-tied rep. Pairing rep-for-rep, gate-ON beats gate-OFF on reps 1–3 and ties on rep 4 (3 non-tied pairs, sign test $p = 0.125$).

Rep	Cold ON	Warm ON	Δ ON	Cold OFF	Warm OFF	Δ OFF
1	5/12	4/12	−1	4/12	0/12	−4
2	6/12	4/12	−2	5/12	0/12	−5
3	7/12	0/12	−7	8/12	0/12	−8
4	6/12	0/12	−6	6/12	0/12	−6
Mean	6.0	2.0	$−4.0 \pm 2.94$	5.75	0.0	$−5.75 \pm 1.71$

Table 11 reports per-task warm-pass results across the three Gemini 3 Flash seeds of the cold→warm monotonicity experiment (§4.2, Table 7; seed totals 11/11/10 match that table exactly). The stable failure set (BrowserDraw, SystemBrightnessMax, SystemWifiTurnOn) fails across all three seeds; the remaining cross-seed variation falls on three sampling-sensitive tasks (BrowserMaze, CameraTakeVideo, ContactsAddContact), where the action sequence is sensitive to LLM sampling.

Table 11. Per-task warm-pass results across the three Gemini 3 Flash seeds on AndroidWorld official-15 (cold→warm monotonicity experiment, §4.2). Seed totals (11/11/10) match Table 7.

Task	seed=42	seed=100	seed=1337
AudioRecorderRecordAudio	PASS	PASS	PASS
AudioRecorderRecordAudioWithFileName	PASS [†]	PASS	PASS [†]
BrowserDraw	FAIL	FAIL	FAIL
BrowserMaze	PASS	PASS	FAIL
CameraTakePhoto	PASS	PASS	PASS
CameraTakeVideo	FAIL	FAIL	PASS
ClockStopWatchPausedVerify	PASS	PASS	PASS
ClockStopWatchRunning	PASS	PASS	PASS
ContactsAddContact	PASS [†]	PASS [†]	FAIL
ContactsNewContactDraft	PASS	PASS	PASS
FilesDeleteFile	PASS	PASS	PASS
MarkorCreateFolder	PASS [†]	PASS [†]	PASS [†]
MarkorCreateNote	PASS	PASS	PASS
SystemBrightnessMax	FAIL	FAIL	FAIL
SystemWifiTurnOn	FAIL	FAIL	FAIL
Total	11/15	11/15	10/15

[†] Live evaluator passes but the verify-before-store gate rejected the task’s recompiled program on that seed (it failed re-verification); the warm run still passed via RPA/hybrid/CUA.

The complete per-task results across all 32 Android multi-seed runs and 8 OSWorld runs are also included in the project’s released artifacts.

Appendix C. Container Patch

Our container patch adds a state-reporting endpoint to the AndroidWorld image that returns a base64-encoded screenshot plus the serialized accessibility-tree elements on demand, sidestepping the upstream populated-AVD a11y-gRPC initialization wedge.

Appendix D. Out-of-Distribution Generalization (Details)

To test whether the corpus transfers beyond seen tasks, we built a corpus from the OSWorld test_tiny set (6 tasks across Chrome, LibreOffice Calc, LibreOffice Writer; 4 verify-gate-stored programs after compile filtering) and ran on the 33 OSWorld test_small tasks that are *not* in test_tiny, spanning 9 domains (chrome $\times$ 2, gimp $\times$ 2, libreoffice_calc $\times$ 1, libreoffice_impress $\times$ 2, multi_apps $\times$ 17, os $\times$ 2, thunderbird $\times$ 2, vlc $\times$ 2, vs_code $\times$ 3). The agentic Program Selector retrieves from the test_tiny corpus when the new task’s description plausibly matches.

Result at $n=3$ reps (corpus copy reset per rep): warm-OOD 21/30, 15/30, 14/30 = **mean** 16.7/30 = 55.6% $\pm$ 12.0. The cold-OOD baseline on the same 30 OOD tasks is a single ($n=1$) pass at approximately 20/30 = 66.7%, so warm-OOD is roughly 11 percentage points *below* cold-OOD (the cold side is unpaired and not replicated, so the magnitude is indicative).

The corpus does not transfer cleanly across task families. Inspecting the rep logs: the selector retrieves a near-miss program (a test_tiny Chrome history-clean program retrieved for a Thunderbird email task whose intent shares lexical surface with “clean”) on $\sim 6/30$ queries per rep on average; the source-domain XPath/state predicates then fail on the OOD page, consuming replay budget that the harness can no longer recover within the per-task wall budget. The remaining $\sim 24/30$ OOD queries are handled correctly by the selector returning “no candidate”—these run as pure CUA and approximately match cold-OOD on their own—so the aggregate 11 pp regression is concentrated in the small fraction of mis-routed retrievals. *The corpus is not neutral on OOD; it is mildly harmful.* This sharpens the scope of the self-extending corpus claim: corpora built on small in-distribution task subsets should not be deployed cross-domain without retraining the selector or extending the corpus to cover the new domain.

Appendix E. Limitations (Extended)

(L1) Architectural baseline. We ran an ActionEngine-style flat-script baseline at $n=5$ Android seeds (Table 14) by disabling per-state verification so that stored programs run as flat scripts. Flat-script mean (warm) is 10.6 ± 1.14 across seeds 42/100/1337/7/2025, vs. state-machine 11.67 ± 0.58 on the matched 3 seeds; the matched-seed $\Delta = +0.67$ tasks in favor of state-machine, sign-test $p = 0.125$ on $n=3$ paired comparisons (suggestive but not significant). The architectural commitment matters but is a smaller effect than the verify-gate’s 1.75–2.6-task diff-of-deltas, consistent with the paper’s framing that the gate is the bigger empirical contribution. We did not run an untargeted-exploration baseline. The Gemini-vs-Claude compile-LLM concern previously flagged here is now closed at $n=3$ (§4.5).

(L2) Benchmark coverage. OSWorld test_tiny has only 6 tasks, AndroidWorld official-15 has 15, WebArena shopping_admin subset has 12. While the verify-gate effect replicates

cross-platform, the benchmarks themselves are a small sample of computer-using tasks. We acknowledge platform-coverage bias as out-of-scope for this paper. The relative percentage-point magnitudes are influenced by the denominator: 43 pp on OSWorld’s 6-task subset is mathematically larger than 17 pp on Android’s 15-task subset and 15 pp on WebArena’s 12-task subset, for absolute task counts that fall in a similar range (1.75–2.6 tasks).

(L3) Compile cost. The verify-before-store gate requires a full re-replay and re-evaluation of every compiled program. From timing measurements across our experiments (60 OSWorld tasks; 60 Android tasks under gate-ON), the verify-replay phase adds median 141 s on OSWorld and 76 s on Android per stored task, on top of the original CUA execution (median 65 s OSWorld; 47 s Android). This corresponds to **+217% wall overhead on OSWorld and +162% on Android per successful CUA-then-compile cycle** (verify-replay is a near-fixed cost roughly comparable to or larger than the original CUA execution on both platforms; it is proportionally largest on OSWorld, where both the absolute verify-replay time and the base CUA time are highest). The cost is amortized on warm runs where the corpus retrieves already-verified programs in seconds, but the compile-time overhead is real and may matter for high-throughput deployments.

(L4) Compile-fidelity rate. A separate audit of stored programs across platforms (Android: 58 programs; OSWorld: 16; WebArena: 7) found two recurring compile-fidelity risk patterns: missing `navigate_back` on nav-heavy edit/delete tasks (affecting roughly 28% of Android programs), and dynamic `inspect_text/inspect_screenshot` returns on pages whose state evolves between Run 1 and replay (100% of audited WebArena programs use `inspect_screenshot`, fully explaining the 48/48 smoking-gun rate observed under gate-OFF). The compiler itself is an LLM that produces lossy state-machines on a non-trivial fraction of inputs. The verify-gate filters this at storage time, but as the corpus scales, the gate’s verify-replay cost (proportional to compile attempts, not just stored programs) may become a bottleneck. Figure 14 breaks down the Android audit.

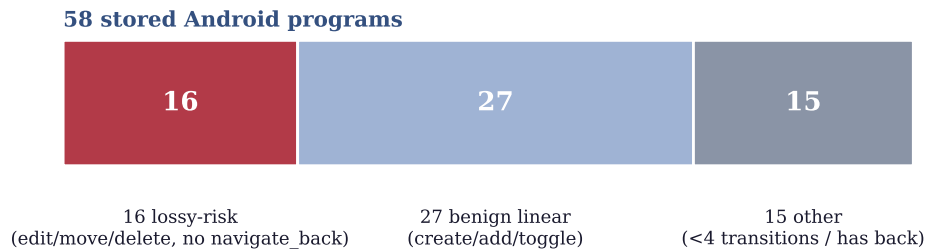


Figure 14. Compile-fidelity audit of the 58-program Android corpus. Roughly a quarter (16 programs) are lossy-risk—nav-heavy edit/move/delete tasks missing a `navigate_back`—which is the rate the gate must catch. Manual inter-classifier agreement was 5/5 per bucket.

(L5) LLM nondeterminism in selector. The agentic Program Selector is itself an LLM call. We measured selector exact-id-match accuracy of $22/30 = 73\%$ on the corpus’s stored programs, prompted with each program’s own task description verbatim. Inspecting the 8 misses, all were picks of *semantically equivalent* programs (same `task_description`, different `program_id` from accumulated reps under different dedup signatures). Functional accuracy (correct task family selected) is therefore close to 100%, with the 73% exact-id figure reflecting the corpus’s accumulation of multiple matches per task.

To stress-test the selector beyond verbatim retrieval, we ran a paraphrase-robustness audit: for each of 15 stored Android programs, we generated 3 LLM-rewritten paraphrases (45 paraphrased queries total) and measured whether the selector still picked the original program. Result: $21/45 = 47\%$ exact-id, $13/45 = 29\%$ same-task-different-id (paraphrase picked an equivalent program), $11/45 = 24\%$ no-pick (selector returned “no candidate”), and **$0/45 = 0\%$ wrong-task-family** (selector never picked an unrelated program). *Functional accuracy* (exact + equivalent) is 75.6% ($34/45$); the no-pick fraction is conservative behavior (the selector errs toward CUA fallback rather than retrieving an inapplicable program). At 75.6% functional retrieval and 0% mis-retrieval, the selector is conservative-by-design rather than over-eager.

Appendix F. Negative-Findings Tables

This appendix provides the per-seed tables for the negative findings summarized in §4.6, including the full selector-backend threshold sweep (Table 12) behind Figure 12.

Table 12. Selector backend ablation (full sweep behind Figure 12). “Functional accuracy” is the fraction of 45 paraphrased Android queries retrieving a program in the correct task family; “No-pick” is the abstention rate on those queries; “False-pick” is the fraction of 15 unrelated-domain queries that wrongly retrieve any program. The wrong-task-family rate on paraphrases is 0% at every operating point. Bold rows mark each backbone’s optimal τ .

Selector	τ	Functional accuracy	No-pick (paraphrase)	False-pick (unrelated)
Embedding MiniLM-L6-v2	0.40	100% (45/45)	0%	6.7% (1/15)
Embedding MiniLM-L6-v2	0.50–0.65	100% (45/45)	0%	0%
Embedding MiniLM-L6-v2	0.70	86.7% (39/45)	13.3%	0%
Embedding MiniLM-L6-v2	0.85	64.4% (29/45)	35.6%	0%
Embedding bge-large-en-v1.5	0.50	100% (45/45)	0%	93.3% (14/15)
Embedding bge-large-en-v1.5	0.60	100% (45/45)	0%	20.0%
Embedding bge-large-en-v1.5	0.65–0.85	100% (45/45)	0%	0%
Agentic LLM (default)	—	75.6% (34/45)	24%	0%

The interpretation: the hypothesis we entered with—“embedding-RAG is inferior because discrimination is a reasoning problem”—does not survive. With either backbone tuned to its safe operating point, embedding retrieval matches or beats the agentic selector at zero wrong-task and zero false-pick; bge-large holds 100% over a wider τ window (0.65–0.85) than MiniLM (0.50–0.65), which matters as the corpus grows and intra-cluster similarity rises. We retain the agentic selector only for its interpretable logs and tuning-free operation, and flag that selectors should be re-tested at 10^3+ programs where these properties become consequential.

Appendix G. Smoking-Gun Reproducibility (Per-Program)

The five smoking-gun $\text{cov}=100\%/\text{score}=0$ cases (Table 3) are not equally reproducible across reps. Table 15 reports the per-program reproducibility across the 5 OSWorld warm-OFF reps.

The mechanism is partially deterministic. For the most reliable case (the Calc formula program), the lossy program reproducibly fails at $\text{cov}=100\%$ on every warm rep. For the

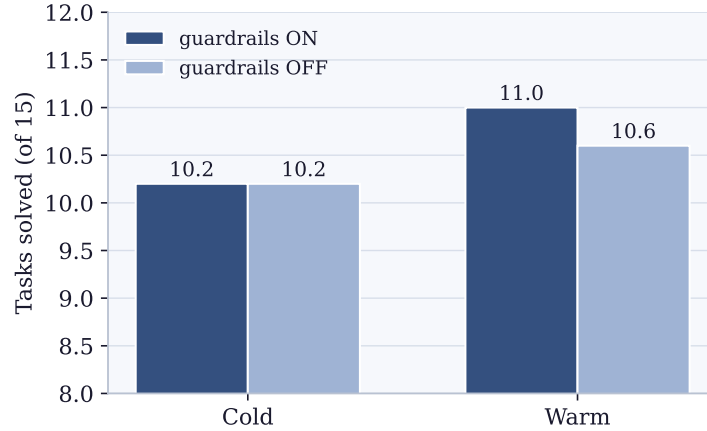


Figure 15. Runtime guardrails are aggregate-neutral. Cold means are identical with and without guardrails ($10.2 = 10.2$); the warm gap ($+0.4$) is well within seed variance. Detail in Table 13.

Table 13. Code-level guardrails ablation, AndroidWorld official-15, $n=5$. Cold means are *identical* ($10.2 = 10.2$). Warm $\Delta = +0.4$ is well within the standard deviations on each side.

Seed	Cold ON	Warm ON	Δ ON	Cold OFF	Warm OFF	Δ OFF
42	10	11	+1	10	12	+2
100	11	11	0	11	10	-1
1337	9	11	+2	10	10	0
2024	12	11	-1	10	10	0
7777	9	11	+2	10	11	+1
Mean	10.2	11.0	$+0.8 \pm 1.30$	10.2	10.6	$+0.4 \pm 1.14$

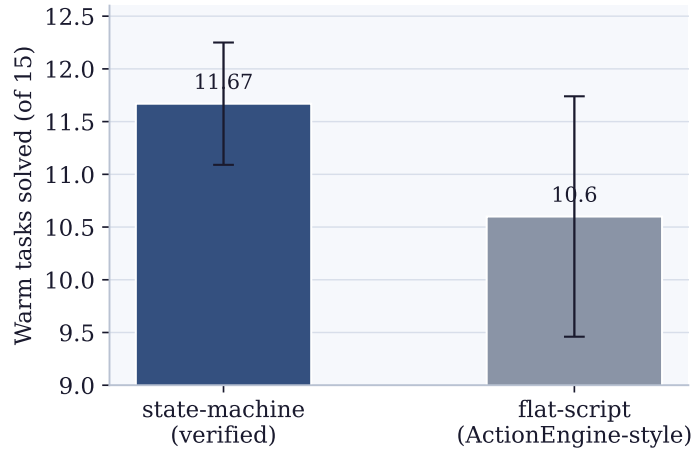


Figure 16. State-machine vs. flat-script runtime (warm SR, AndroidWorld official-15). The verified graph form helps by a small, consistent margin ($+0.67$ tasks, all three matched seeds non-regressive, $p = 0.125$)—real but far smaller than the gate’s 1.75–2.6 tasks.

Calc chart program, hybrid-mode rescue occasionally salvaged the run via CUA fallback after partial replay (cov=8%). For the Chrome history program, the cold-stored program’s mechanical action sequence happened to match the evaluator’s required browser state on 3

Table 14. Architectural ablation: state-machine vs. flat-script runtime, AndroidWorld official-15 warm SR. State-machine $n=3$ (matched seeds), flat-script $n=5$ (matched seeds + 7 + 2025).

Mode	seed=42	seed=100	seed=1337	seed=7	seed=2025	Mean $\pm$ std
state_machine (default)	12/15	12/15	11/15	—	—	11.67 $\pm$ 0.58 ($n=3$)
flat_script (ActionEngine-style)	11/15	12/15	10/15	11/15	9/15	10.6 $\pm$ 1.14 ($n=5$)
Δ (matched)	+1	0	+1	—	—	+0.67 (matched $n=3$)

Table 15. Smoking-gun reproducibility across 5 OSWorld warm-OFF reps. Each row: how many of 5 reps replayed the program at cov=100% and got score=0.

Task	Reps with cov=100%/score=0
LibreOffice Calc formula	5/5 (every rep)
LibreOffice Calc chart	4/5 (rep2 hybrid-rescued)
Chrome history clean	2/5 (rep3/4/5 replayed correctly)

of 5 reps—a partial-determinism that traces to environmental factors (Chrome’s session state, browsing history initialization) varying across environment resets even with identical task IDs.

Appendix H. Validity-Threat Closure Map

Table 16 maps each validity threat—the original nine from our pre-experiment validation plan, plus three added since the May 2026 revision—to its closure status and the experiment that closes it. Eleven of the twelve are in scope and closed, most with multi-seed runs and the remainder at the rigor noted in the Evidence column (e.g., codebase inspection for threat 4, manual inter-rater agreement for threat 6, $n=3$ for threats 11–12); threat 8 (platform coverage) is out of scope as it would require net-new benchmarks.

Table 16. Validity threats and closure status. Eleven in-scope threats closed at the rigor noted in the Evidence column (most multi-seed); threat 8 is out of scope.

#	Threat	Status	Evidence
1	Single-run variance	Closed	$n=3$ cold $\rightarrow$ warm + $n=5$ cold-runs
2	Verify-gate ablation	Closed	cross- platform on 3 platforms Android $n=5$ ($p < 0.001$); OSWorld $n=5$ (43 pp); WebArena $n=4+4$ (≈ 15 pp); meta $p \approx 10^{-4}$ $n=3$ with the corpus reset, all monotonic
3	Android cold $\rightarrow$ warm monotonicity	Closed	
4	Prompt-guidance inertness	Closed (codebase state)	
5	Code-level guardrails	Closed $n=5$	Production CUA path omits the optional prompt guidelines
6	Compile-fidelity taxonomy	Closed	Aggregate-neutral (cold means $10.2 = 10.2$)
7	Step-budget AB	Closed $n=5$	5/5 manual agreement per bucket
8	Platform-coverage	Out of scope	Within ± 2 prediction; wall ratio 0.69
9	Baseline-parity replication	Closed	Requires net-new benchmarks
10	Cache-miss-to-CUA fallback (Muscle-Mem gap)	Closed $n=4$	Cross-model and cross-seed
11	Compile-LLM robustness (Gemini vs Claude)	Closed $n=3$	WebArena warm- $\Delta -4.0 \rightarrow -1.0$, matches Muscle-Mem -0.75 (Welch’s 2-sided $p \approx 0.84$)
12	Embedding-selector robustness (MiniLM vs bge-large)	Closed	WebArena cold $6.0 \approx 6.33$; rejection $83 \approx 89\%$; mechanism shared Both 100% functional retrieval; bge-large wider safe τ window